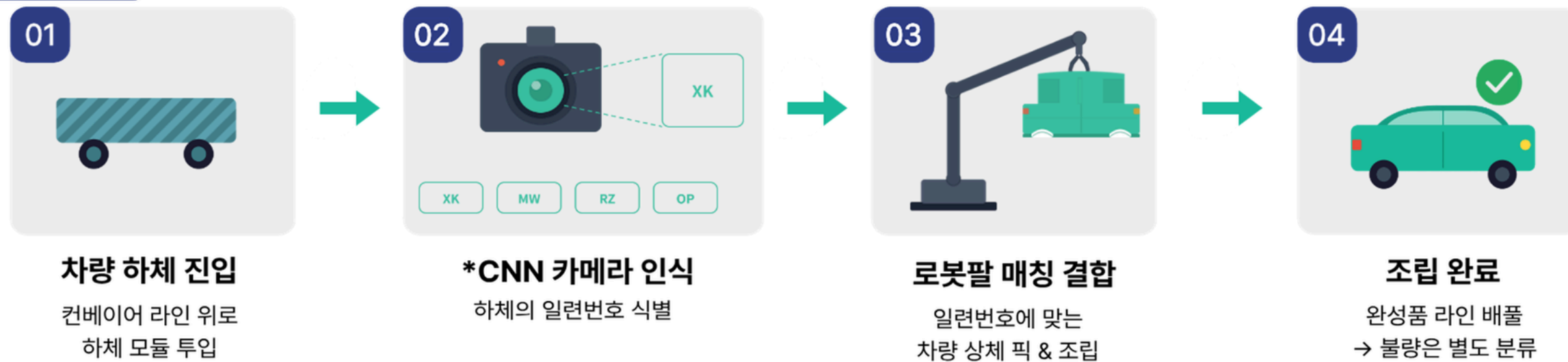


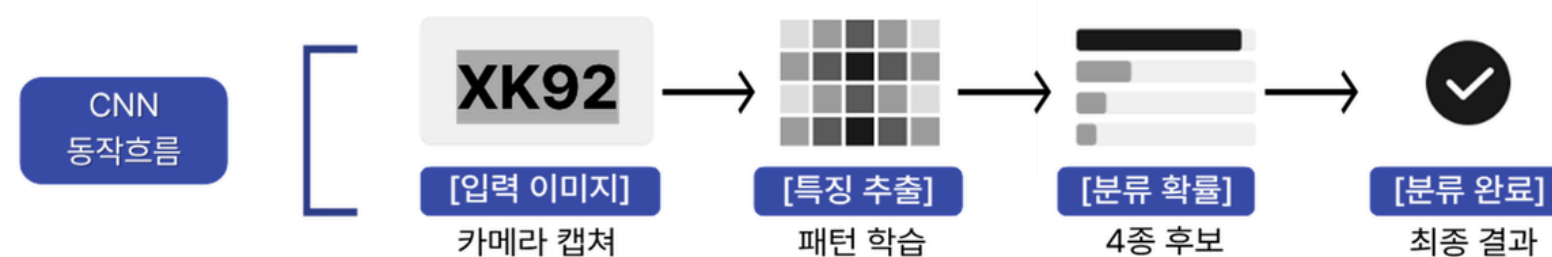
06. 디지털트윈 스마트팩토리

프로젝트 개요

[환경 조성]

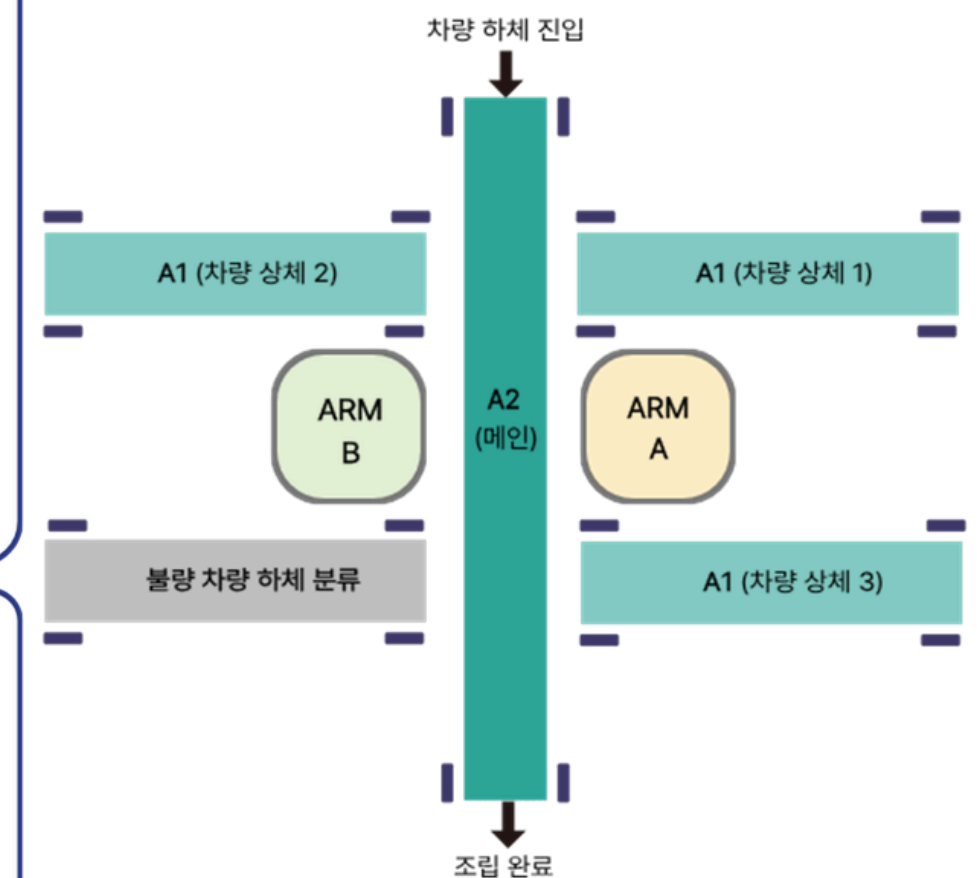


CNN(Convolutional Neural Network)이란? 이미지 패턴을 스스로 학습해 무엇인지 분류하는 딥러닝 모델



본 프로젝트 적용

- 1) 카메라로 차량 하체 촬영
- 2) CNN이 일련번호 인식
- 3) 결과를 로봇팔에 전달



카메라 기반 일련번호 인식 결과를 활용해 조립 공정을 자동화한 스마트팩토리 시스템

개발 환경

Development Tool

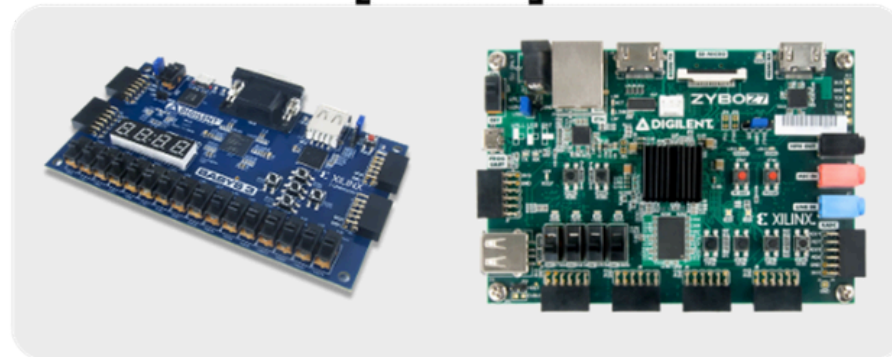


Language



Tool

[FPGA]



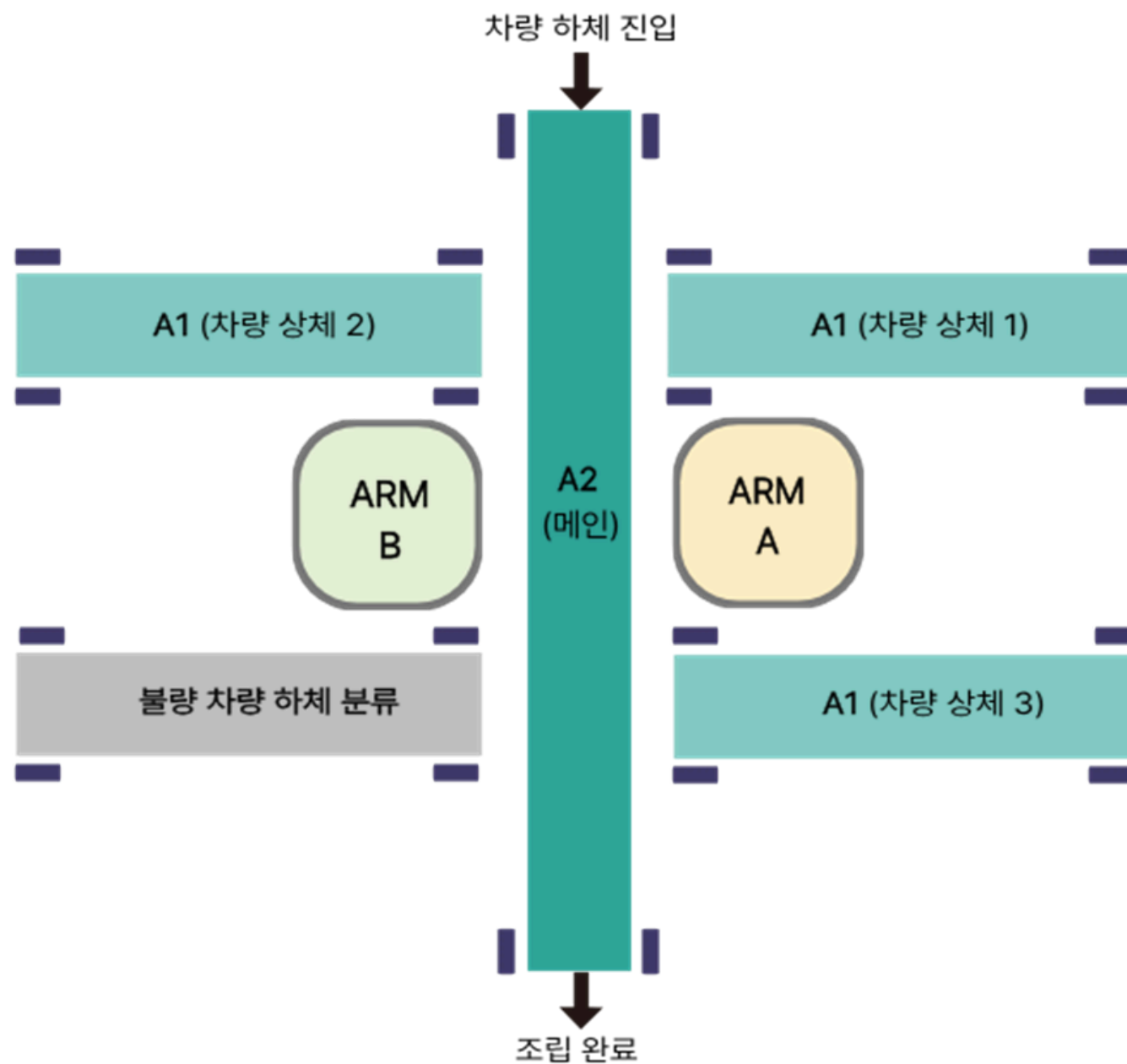
[MCU]



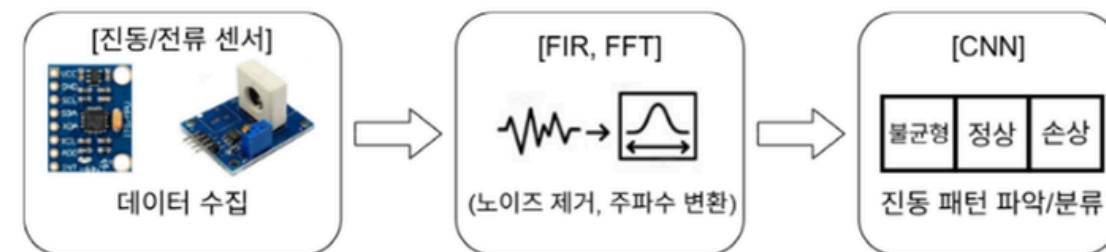
[MPU]



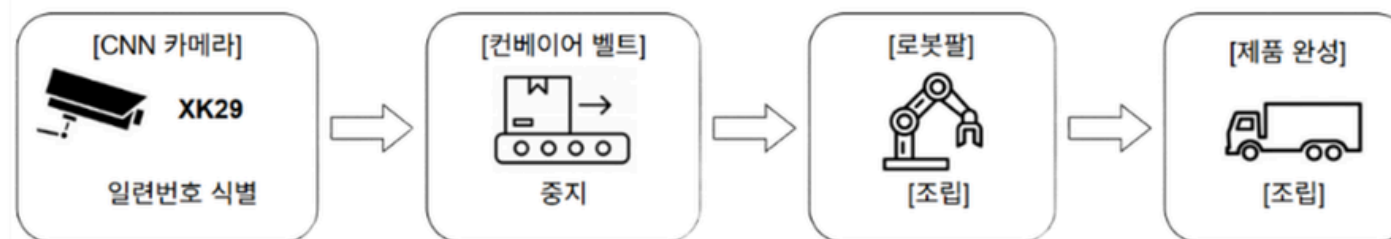
스마트팩토리 동작 흐름



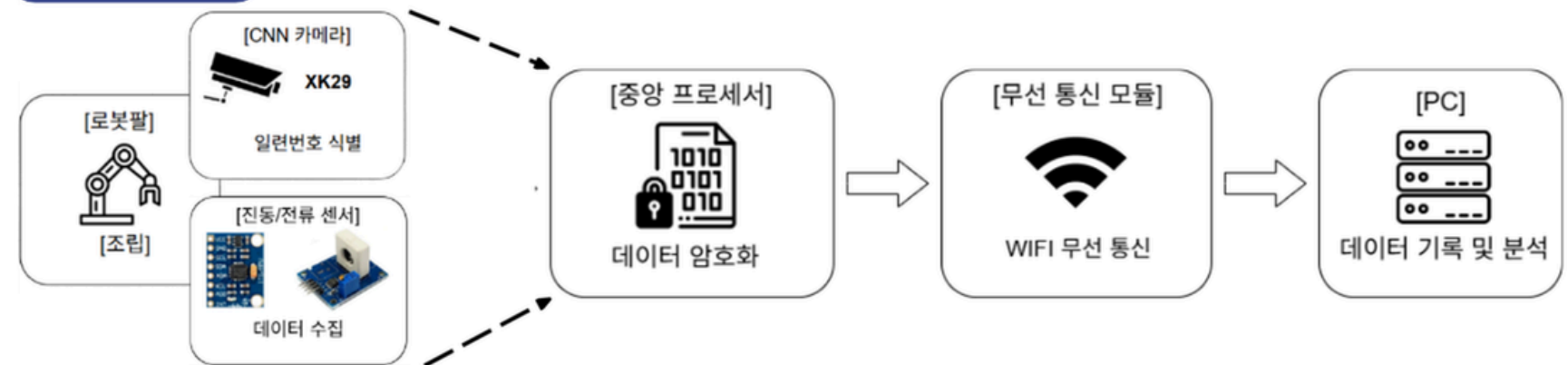
결함 측정 과정



메인 조립 과정

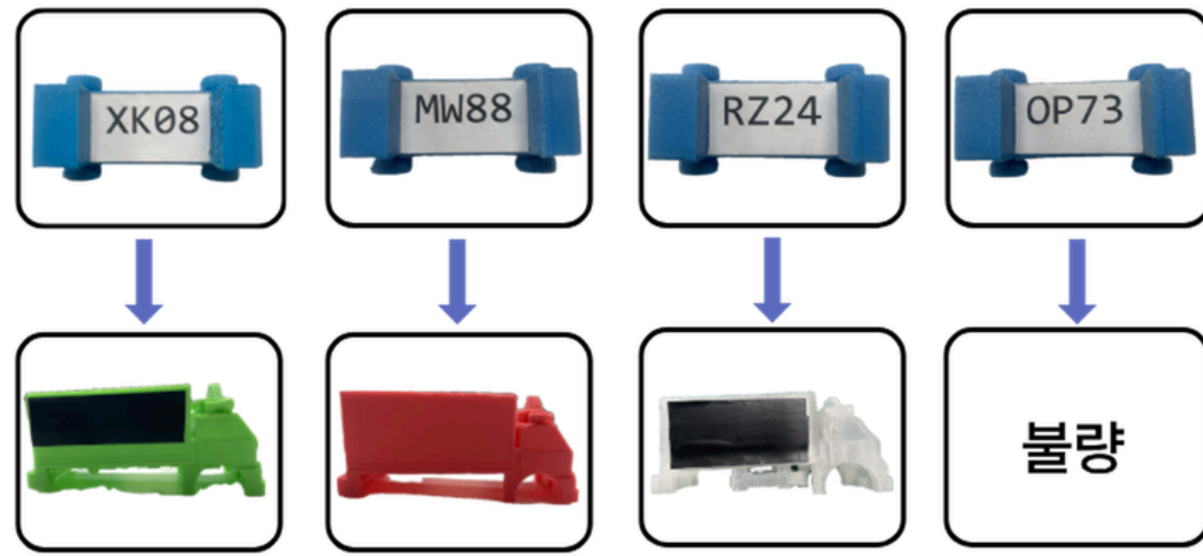


서버 기록 과정



스마트팩토리 동작 흐름

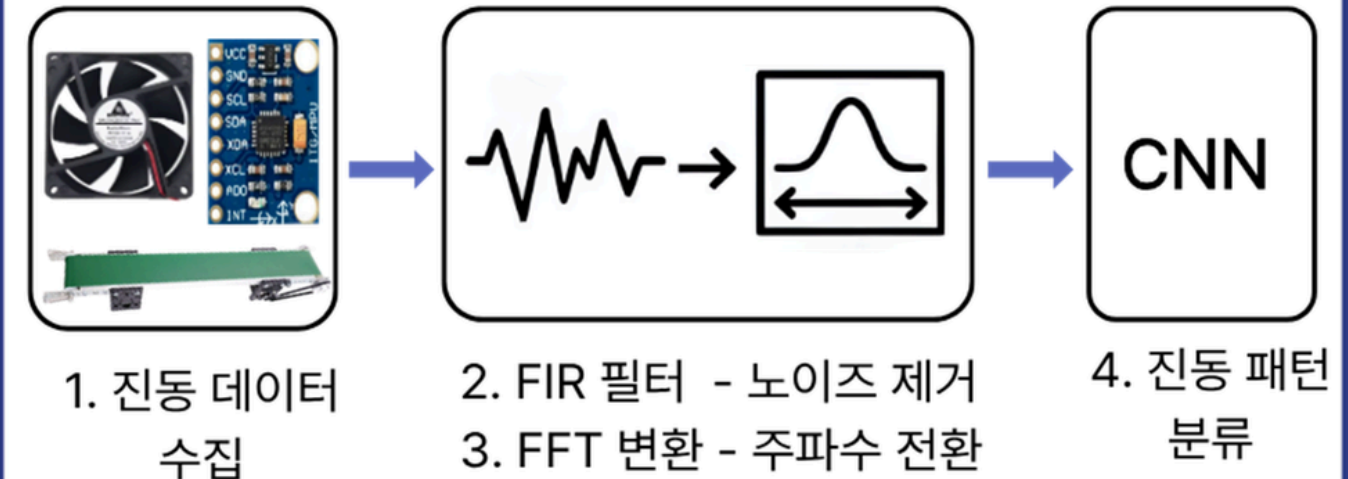
[메인 공정] 01 카메라



[메인 공정] 02 조립



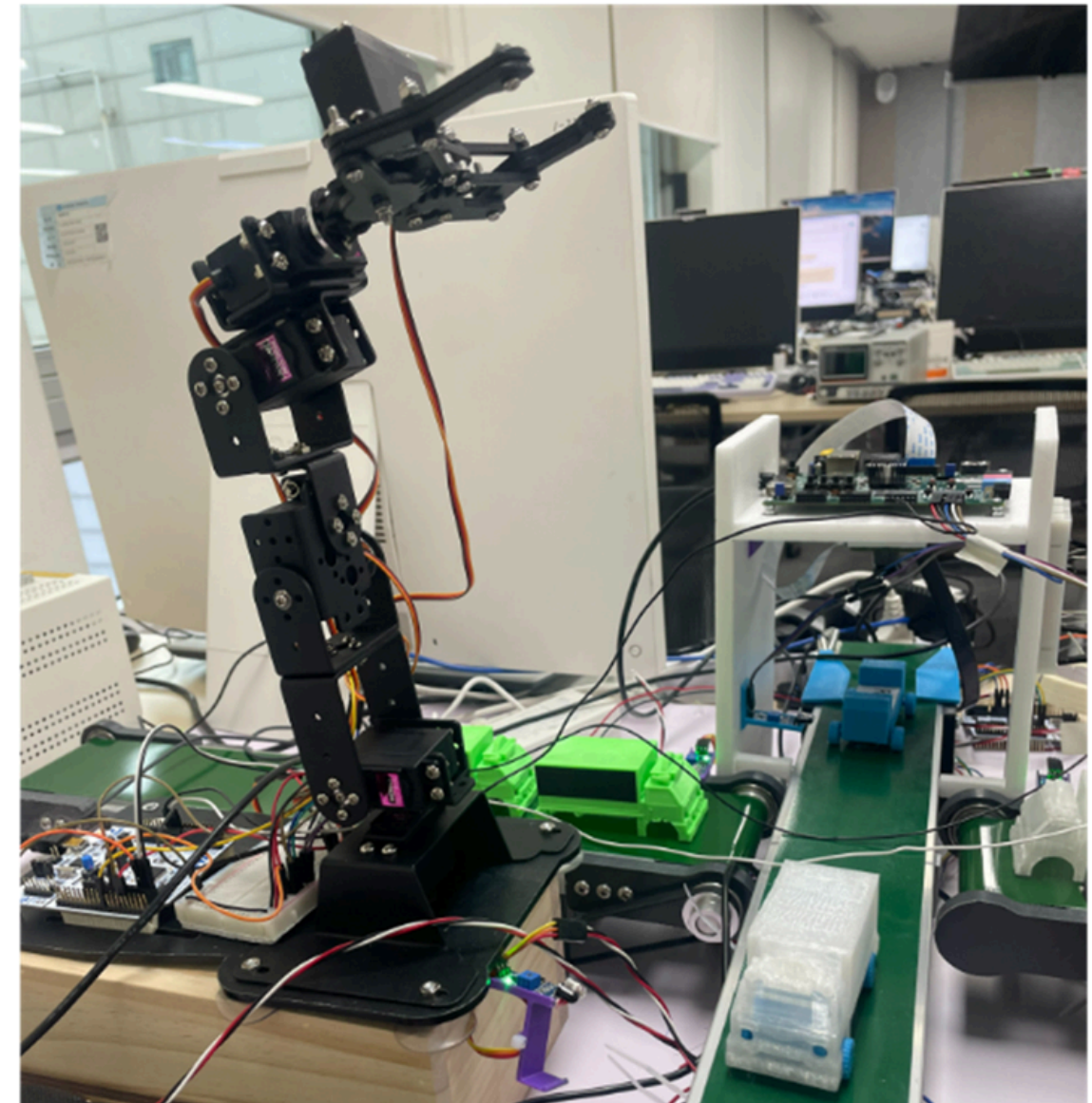
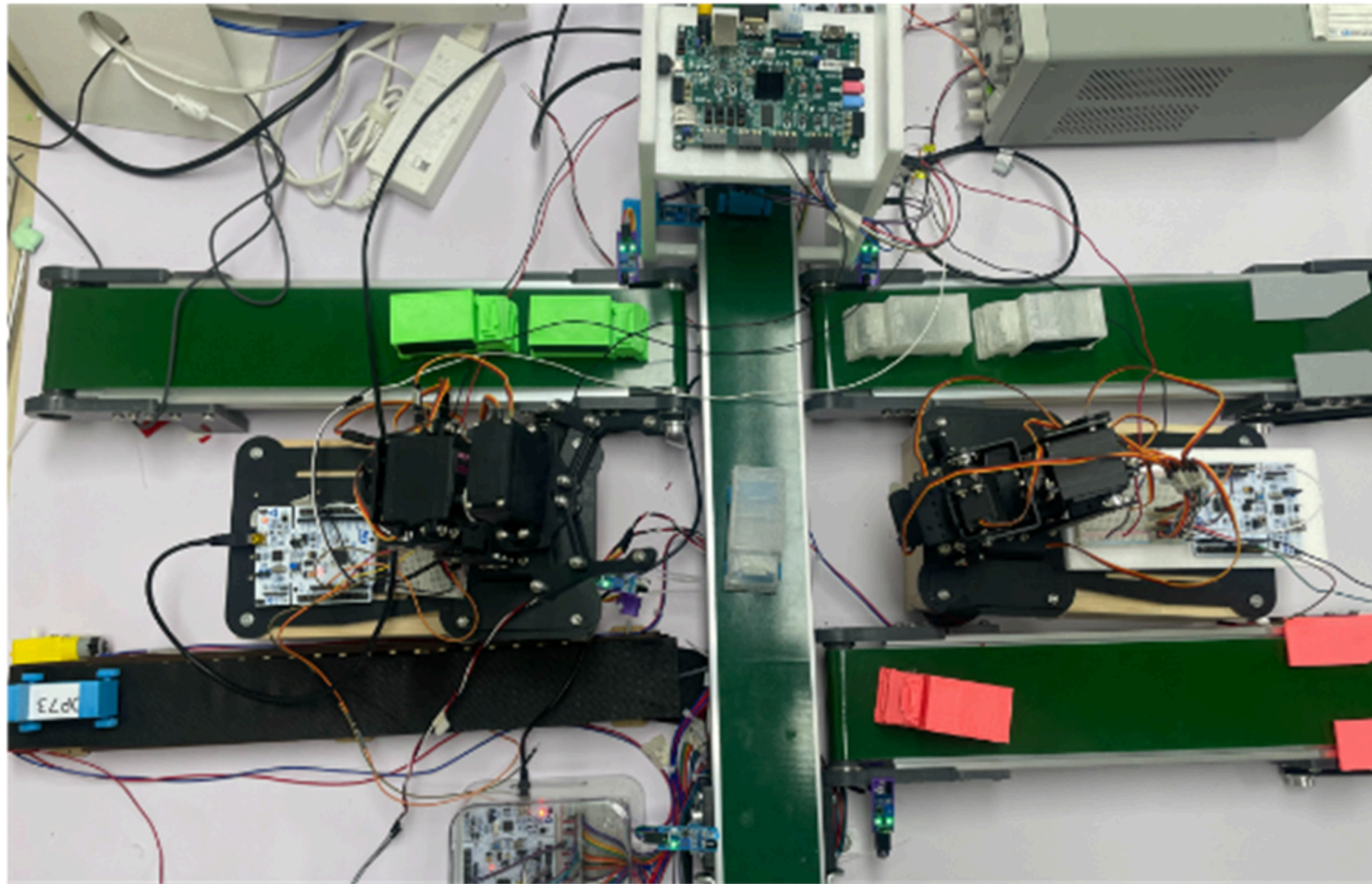
03 설비 측정



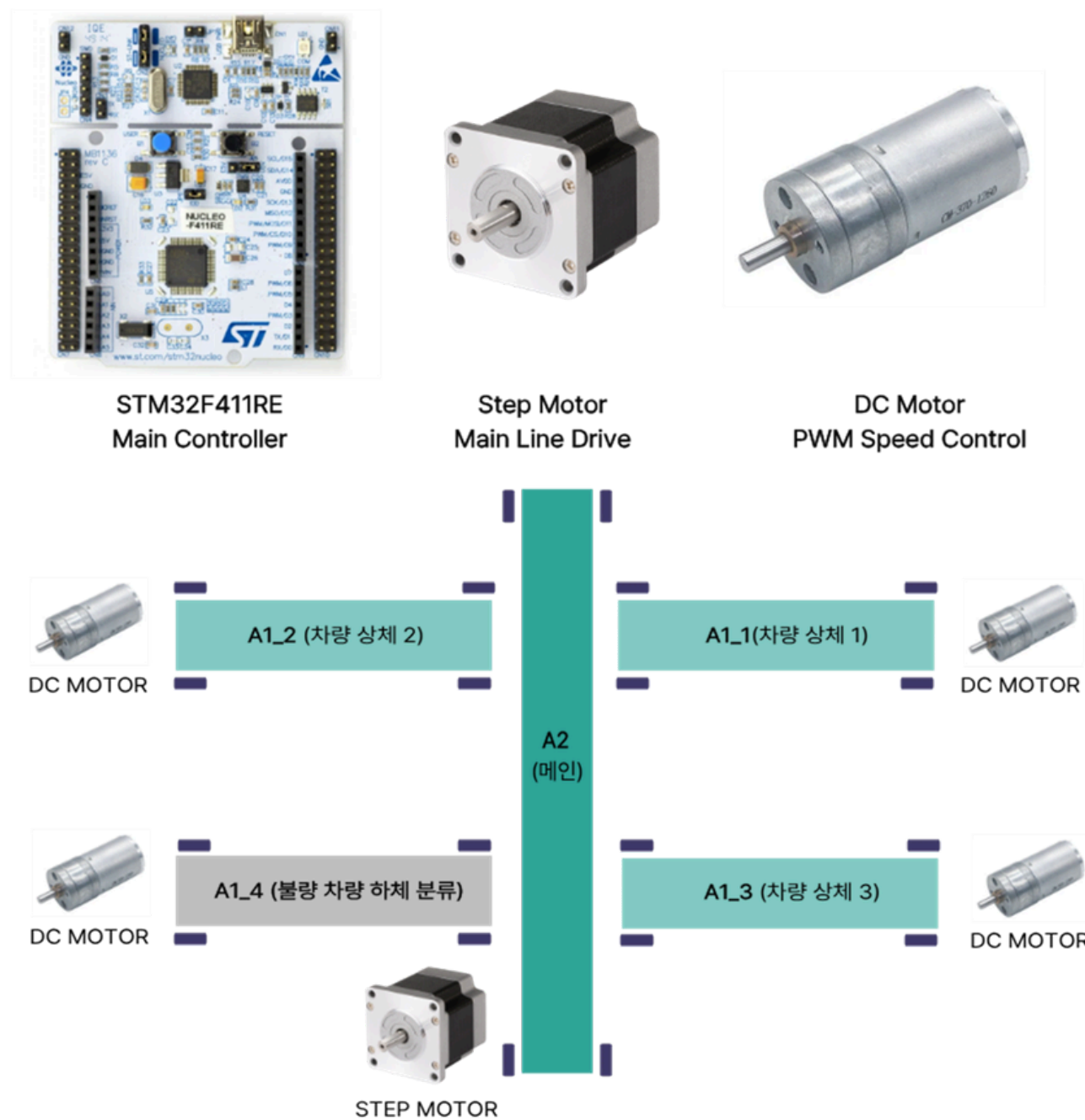
04 서버 통신 및 암호화



HW 구성



컨베이어 벨트 제어



컨베이어	모터	타이머	역할
<u>A2</u>	STEP MOTOR	TIM2 ISR	메인 조립 라인
<u>A1_1</u>	DC MOTOR	TIM3 CH2	서브 상체 1 공급
<u>A1_2</u>	DC MOTOR	TIM3 CH1	서브 상체 2 공급
<u>A1_3</u>	DC MOTOR	TIM4 CH2	서브 상체 3 공급
<u>A1_4</u>	DC MOTOR	TIM4 CH2	불량 차량 하체 분류

STEP MOTOR : TIM2 ISR(Interrupt Service Routine) 기반

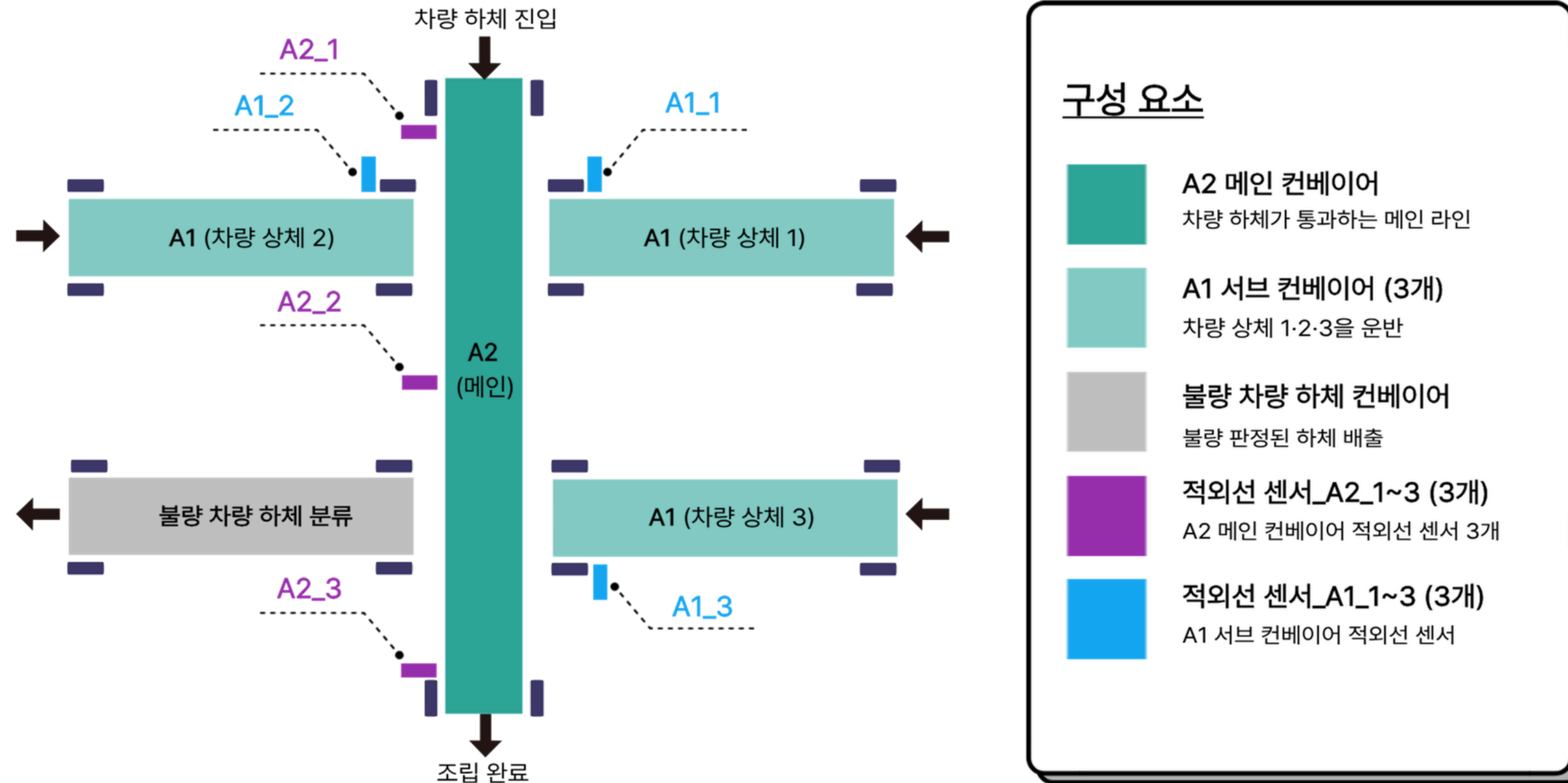
Step Sequence 제어

→ 인터럽트 주기 기반 메인 라인 위치 이동 수행

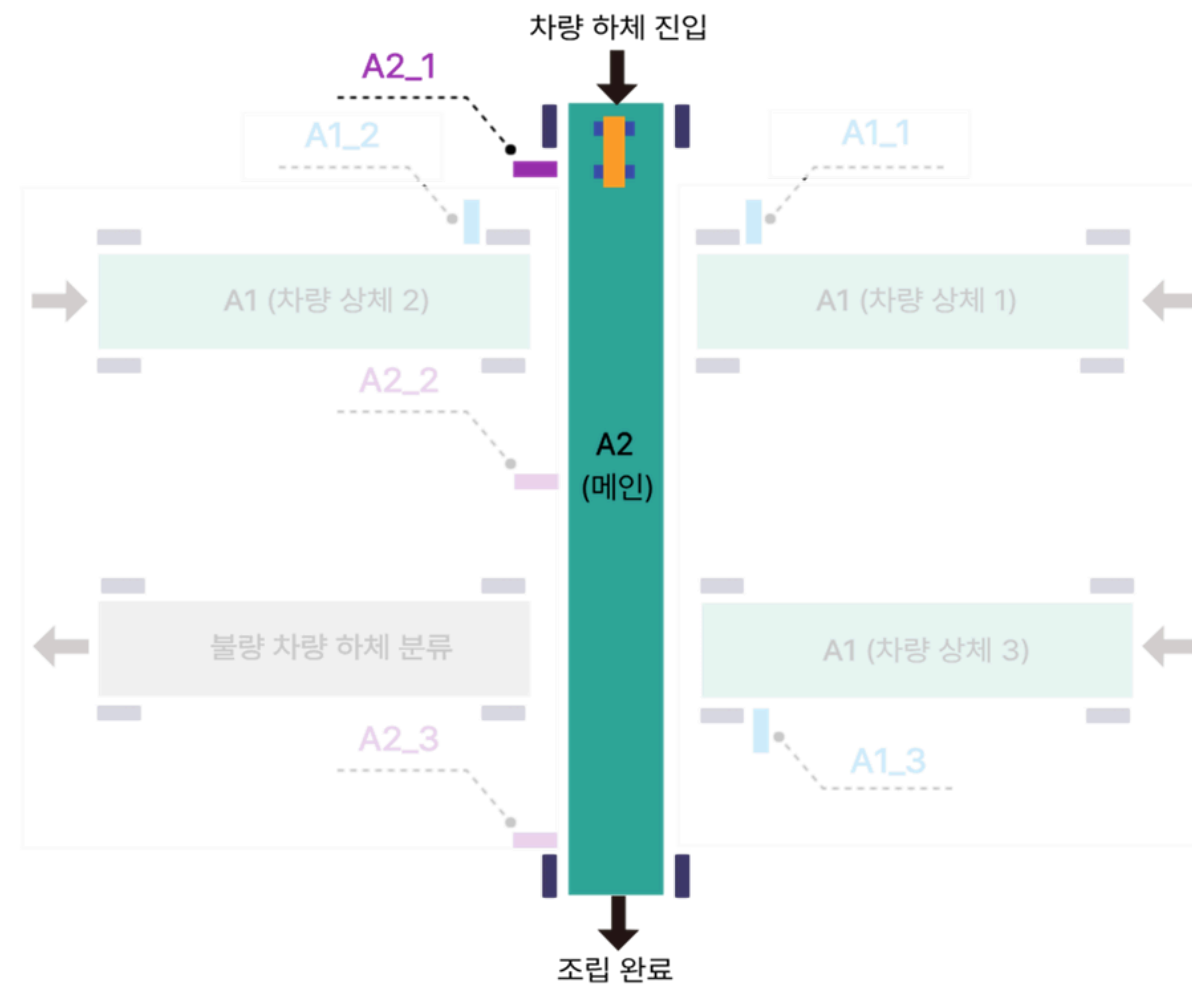
DC MOTOR : TIM3/TIM4 PWM 기반 속도 제어

→ IR Sensor 감지 상태 기반 독립 구동 및 정지 제어

컨베이어 벨트 제어



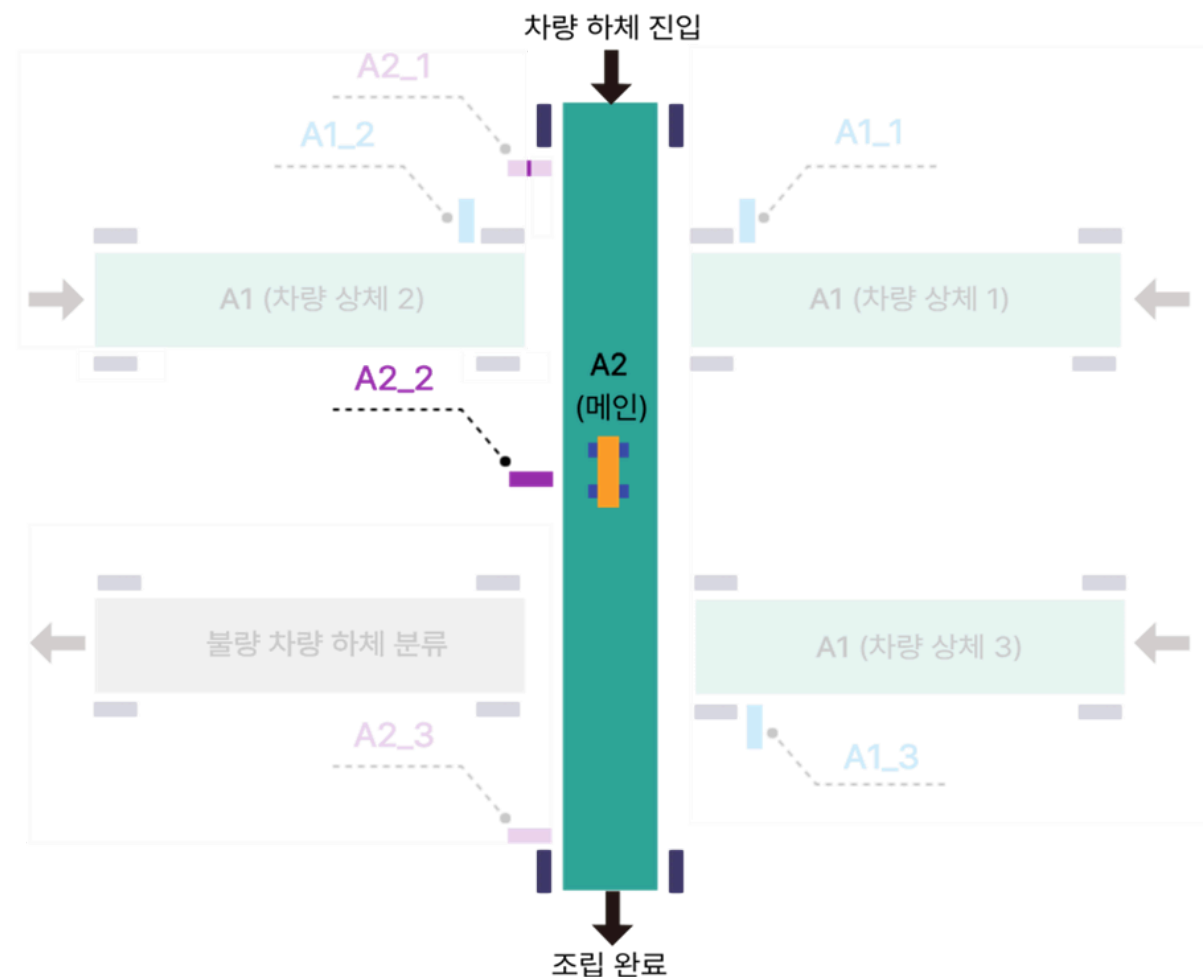
컨베이어 벨트 제어



적외선 센서가 각 컨베이어 벨트 물체를 인식
→ CNN 가속기 동작의 트리거 역할 수행

A2_1 센서	차량 하체	센서	송신
메인 컨베이어에 하체의 등장을 CNN으로 송신			
A2_2 센서	차량 하체	센서	송수신
하체의 조립 위치 도착을 로봇팔에 송신 조립 완료 DONE 신호를 로봇팔로부터 수신			
A2_3 센서	조립 완료	센서	완료
조립이 완료되어 도달하면 끝단 도착을 감지하여 조립라인 중지			
A1_1~3 센서	차량 상체	센서	공급
좌측 차 상체2 컨베이어에서 상체를 픽업 후 메인 컨베이어 하체 위로 옮김			

컨베이어 벨트 제어



적외선 센서가 각 컨베이어 벨트 물체를 인식
→ 컨베이어 정지 및 로봇팔 동작 트리거 역할 수행

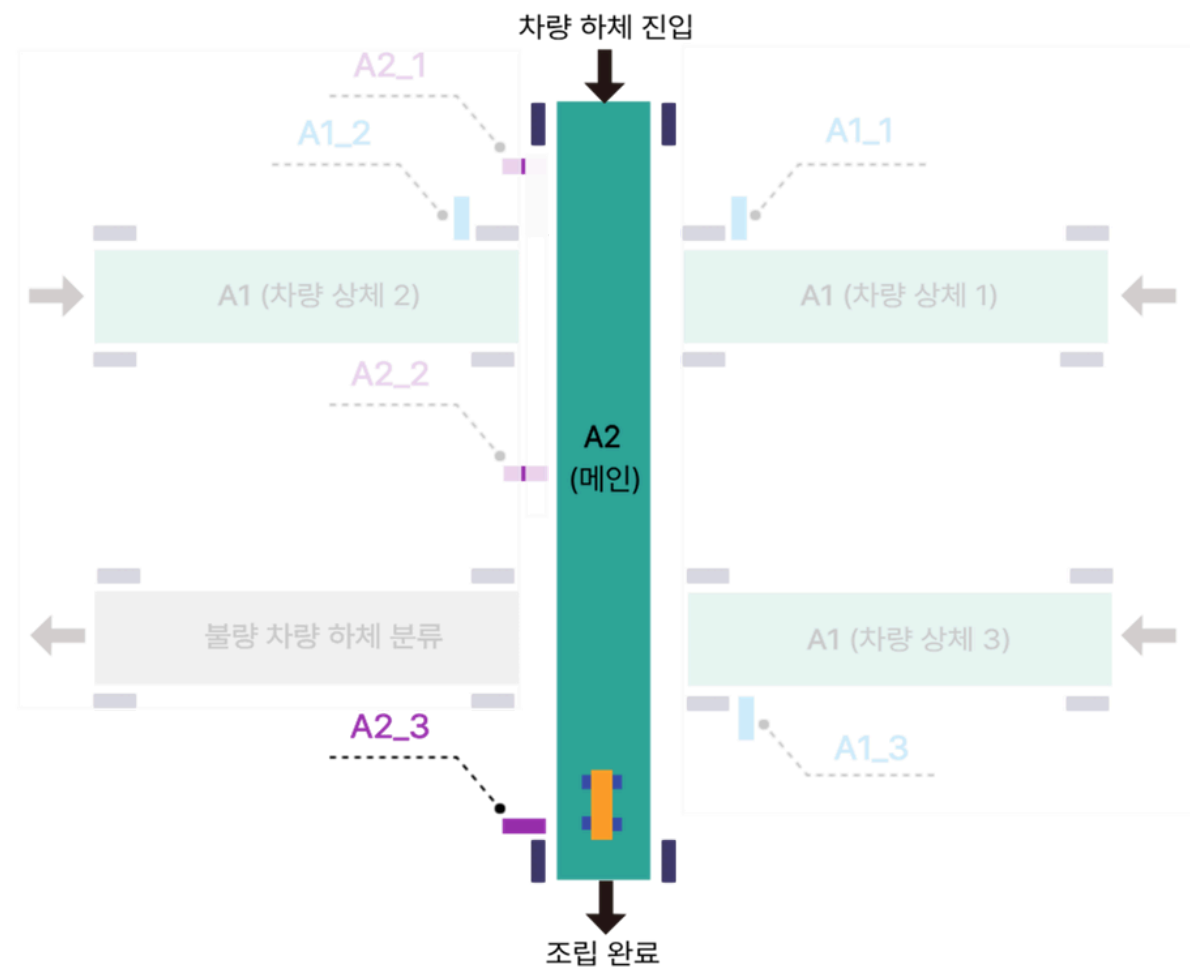
A2_1 센서	차량 하체	센서	송신
메인 컨베이어에 하체의 등장을 CNN으로 송신			

A2_2 센서	차량 하체	센서	송수신
하체의 조립 위치 도착을 로봇팔에 송신 조립 완료 DONE 신호를 로봇팔로부터 수신			

A2_3 센서	조립 완료	센서	완료
조립이 완료되어 도달하면 끝단 도착을 감지하여 조립라인 중지			

A1_1~3 센서	차량 상체	센서	공급
좌측 차 상체2 컨베이어에서 상체를 픽업 후 메인 컨베이어 하체 위로 옮김			

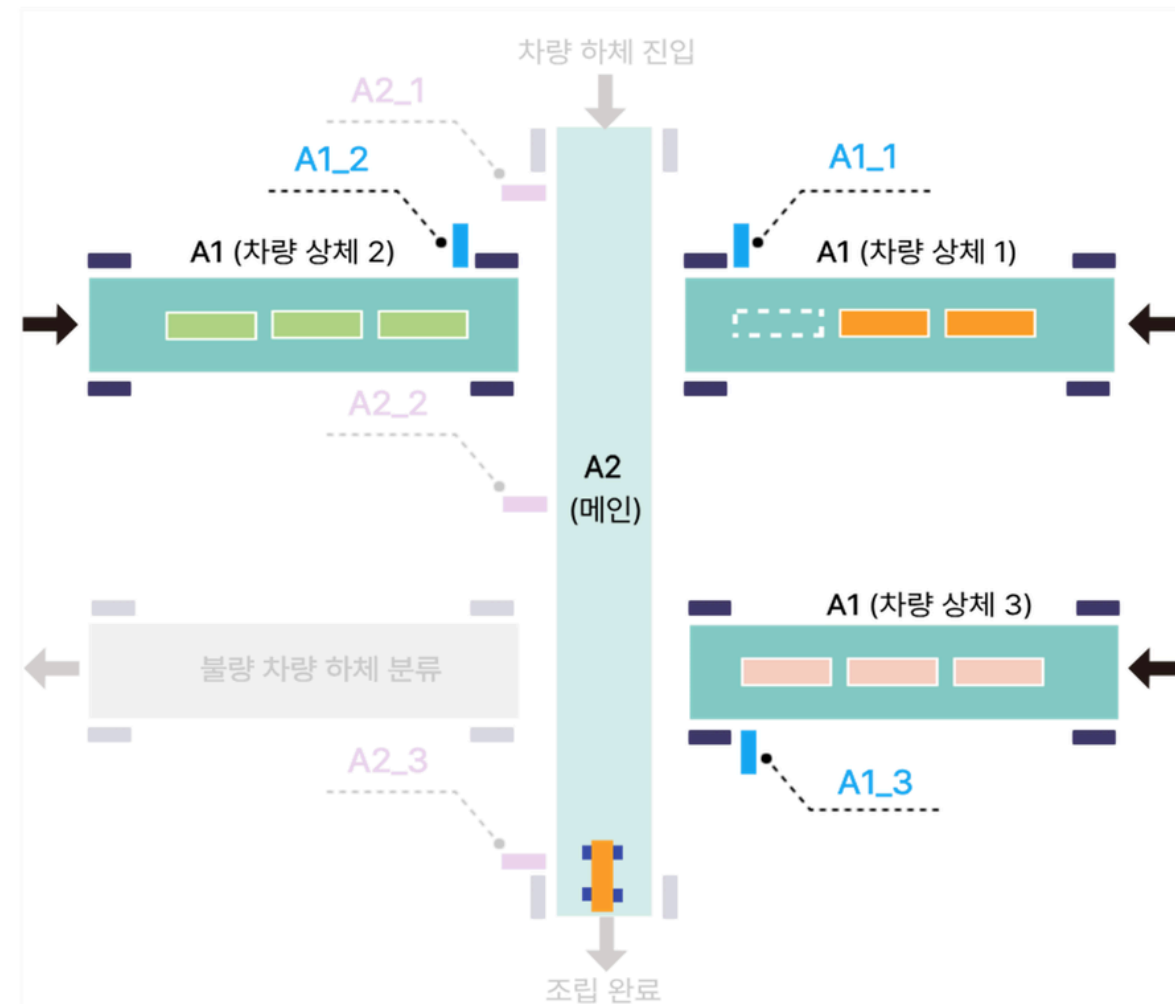
컨베이어 벨트 제어



적외선 센서가 각 컨베이어 벨트 물체를 인식
→ **컨베이어 정지** 역할 수행

A2_1 센서	차량 하체	센서	송신	메인 컨베이어에 하체의 등장을 CNN으로 송신
A2_2 센서	차량 하체	센서	송수신	하체의 조립 위치 도착을 로봇팔에 송신 조립 완료 DONE 신호를 로봇팔로부터 수신
A2_3 센서	조립 완료	센서	완료	조립이 완료되어 도달하면 끝단 도착을 감지하여 조립라인 중지
A1_1~3 센서	차량 상체	센서	공급	좌측 차 상체2 컨베이어에서 상체를 픽업 후 메인 컨베이어 하체 위로 옮김

컨베이어 벨트 제어

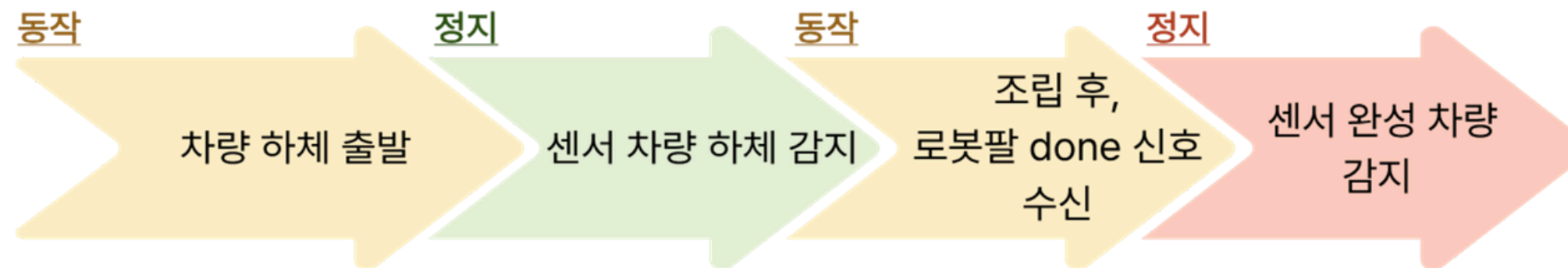


적외선 센서가 각 컨베이어 벨트 물체 유무를 감지
→ 컨베이어 동작으로 **차 본체를 자동 공급**

A2_1 센서	차량 하체	센서	송신
메인 컨베이어에 하체의 등장을 CNN으로 송신			
A2_2 센서	차량 하체	센서	송수신
하체의 조립 위치 도착을 로봇팔에 송신 조립 완료 DONE 신호를 로봇팔로부터 수신			
A2_3 센서	조립 완료	센서	완료
조립이 완료되어 도달하면 끝단 도착을 감지하여 조립라인 중지			
A1_1~3 센서	차량 상체	센서	공급
좌측 차 상체2 컨베이어에서 상체를 픽업 후 메인 컨베이어 하체 위로 옮김			

컨베이어 벨트 제어

메인 컨베이어 벨트

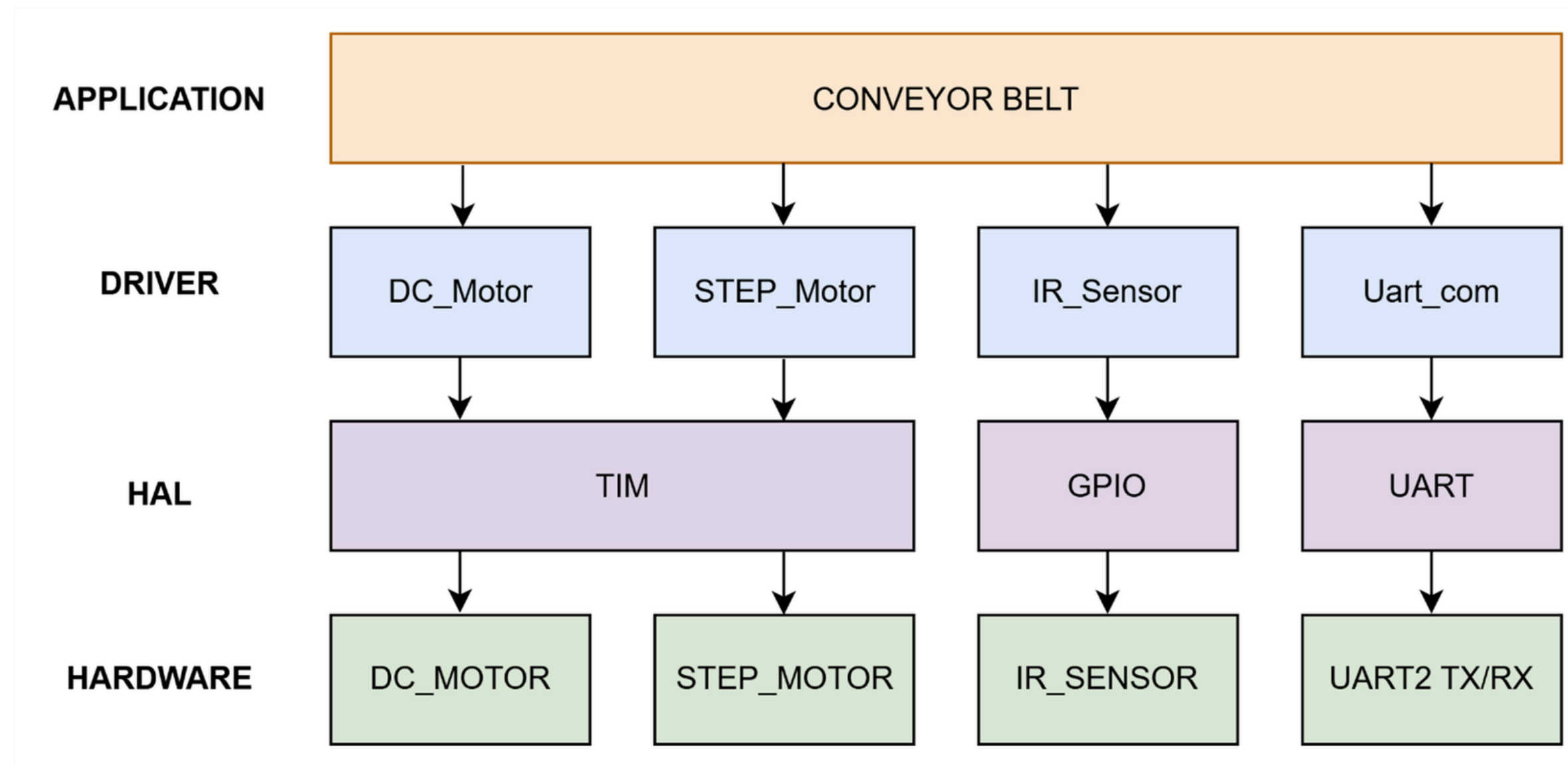


서브 컨베이어 벨트

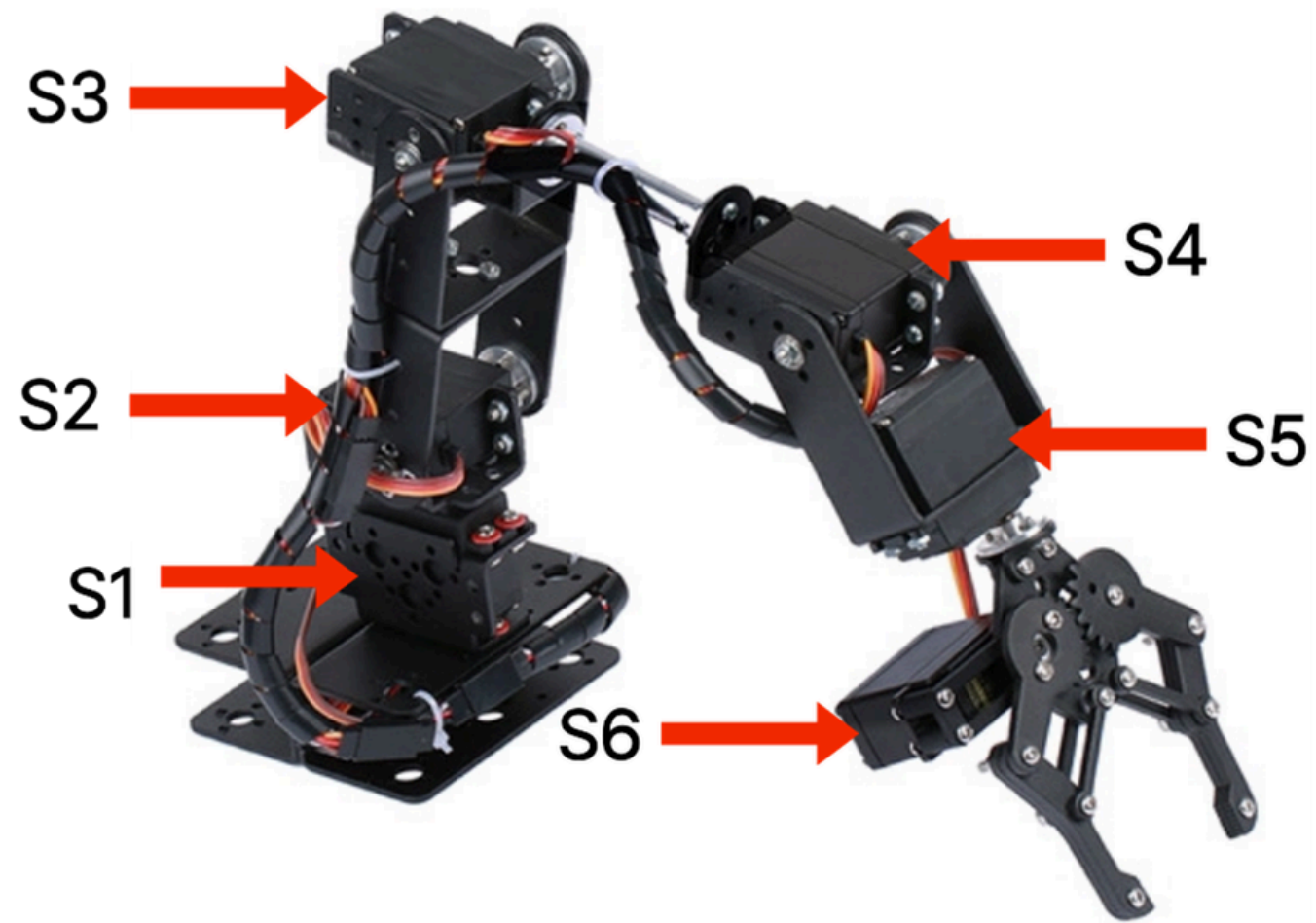


- 메인 컨베이어 벨트 - 로봇팔 동작과 연동되어 제어
 - IR 센서 감지 시 정지 후, Robot Arm DONE 신호 수신 시 재동작
- 서브 컨베이어 벨트 - 타 장치와 독립적으로 동작
 - IR 센서 감지 여부에 따라 자동 정지 / 재동작 제어

컨베이어 벨트 제어



로봇팔 제어

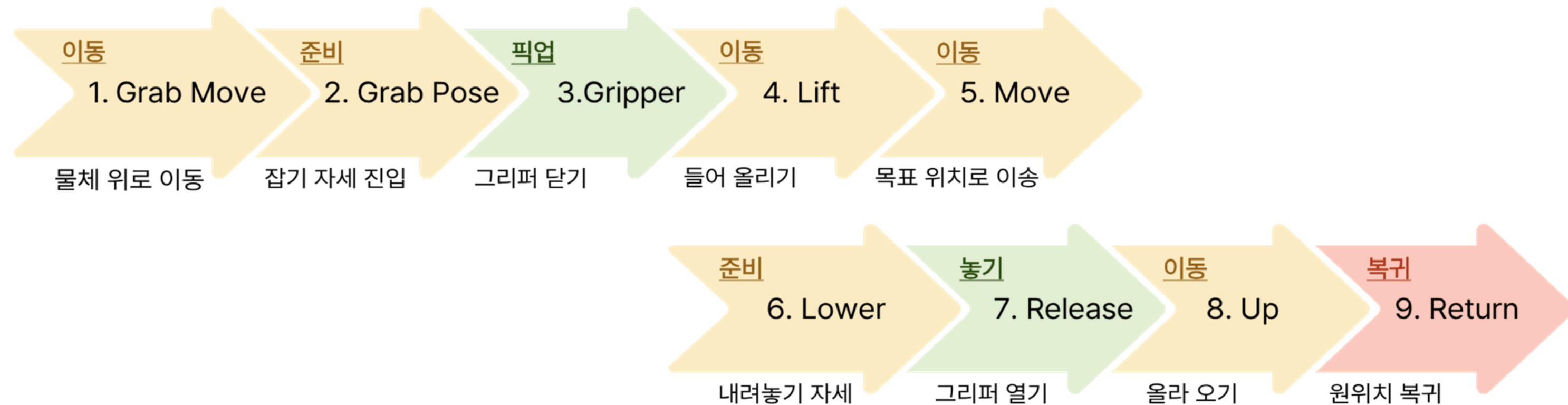


6자 유도 로봇팔

서보	관절	타이머	역할
<u>S1</u>	Base	TIM2 CH1	회전 (좌우)
<u>S2</u>	Shoulder	TIM2 CH2	어깨 상하
<u>S3</u>	Elbow	TIM3 CH1	팔꿈치 굽힘
<u>S4</u>	Wrist	TIM3 CH2	손목 각도
<u>S5</u>	Rotate	TIM4 CH1	그리퍼 회전
<u>S6</u>	Gripper	TIM4 CH2	집게 개폐

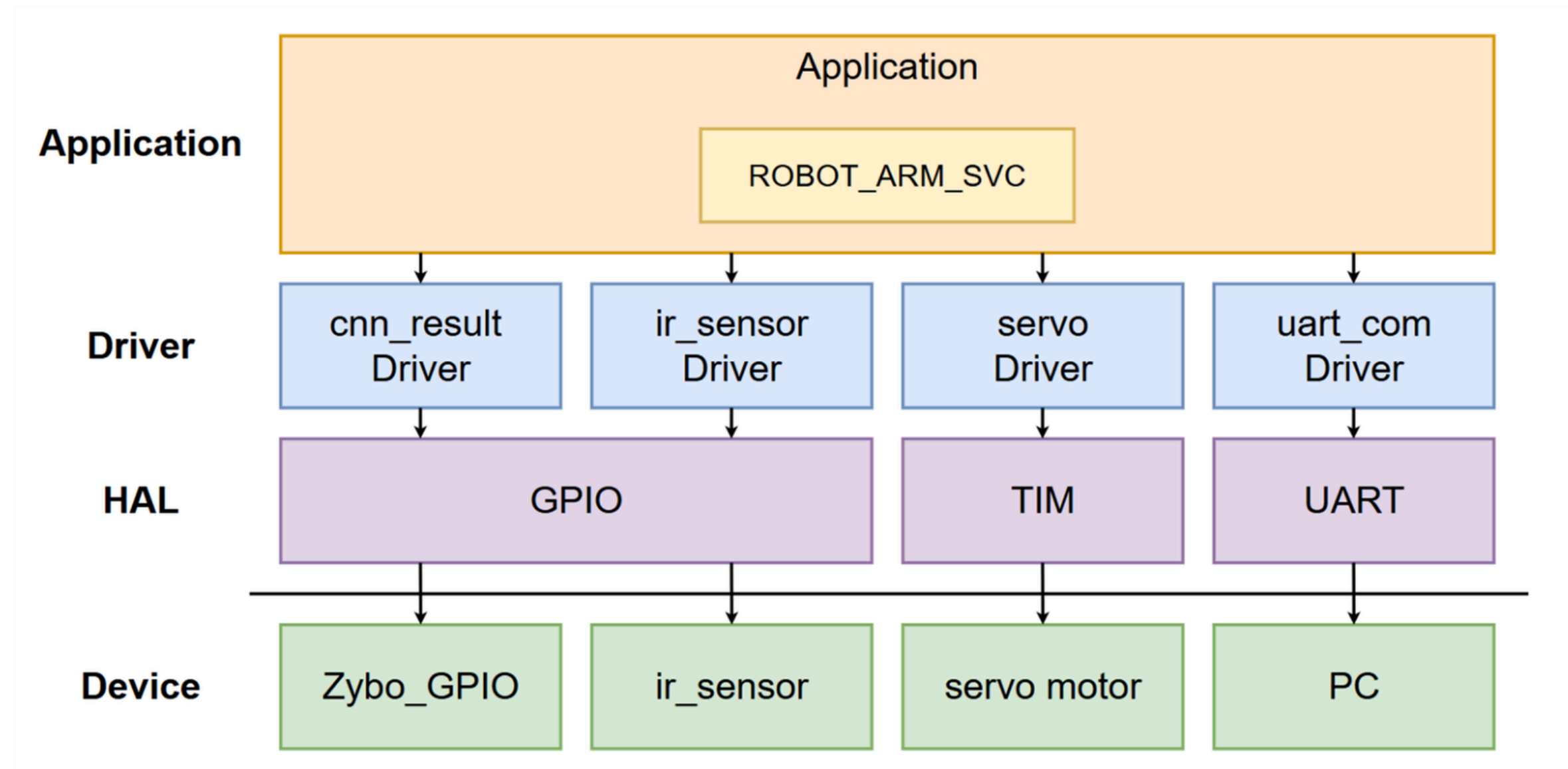
PWM 신호 -> 50Hz / 1~2ms 펄스폭으로 0~360° 위치 제어

로봇팔 제어

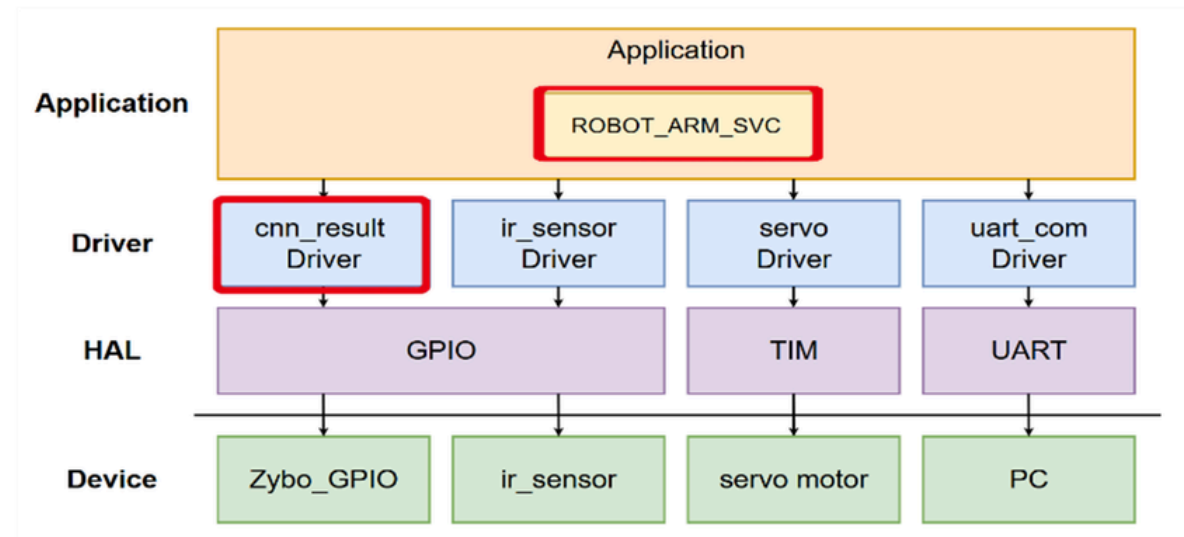


각 단계마다 6개 서보의 목표 각도 [S1, S2, S3, S4, S5, S6] 로 정의
→ UART로 전송
→ STM32가 PWM으로 변환하여 서보 구동

로봇팔 제어



로봇팔 제어

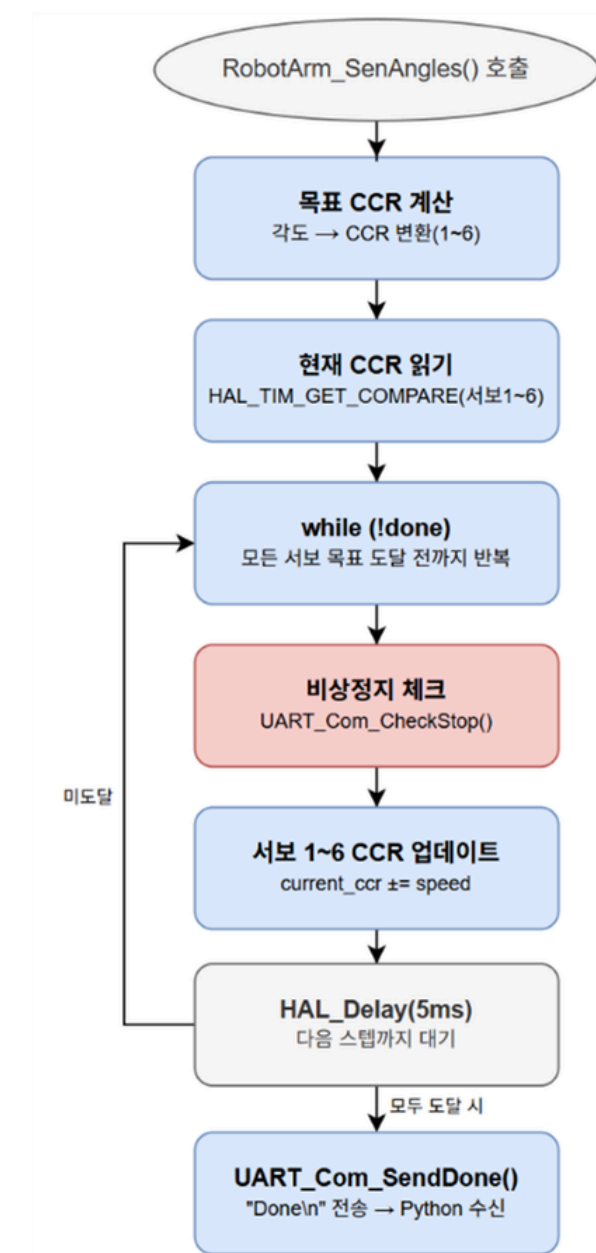


Robot Arm Service

Driver 계층을 조합해 로봇팔 동작 시퀀스를 구현하는 계층.
ap_main에서 움직이라는 신호를 받아 구체적으로 어떻게 움직일지를 담당.

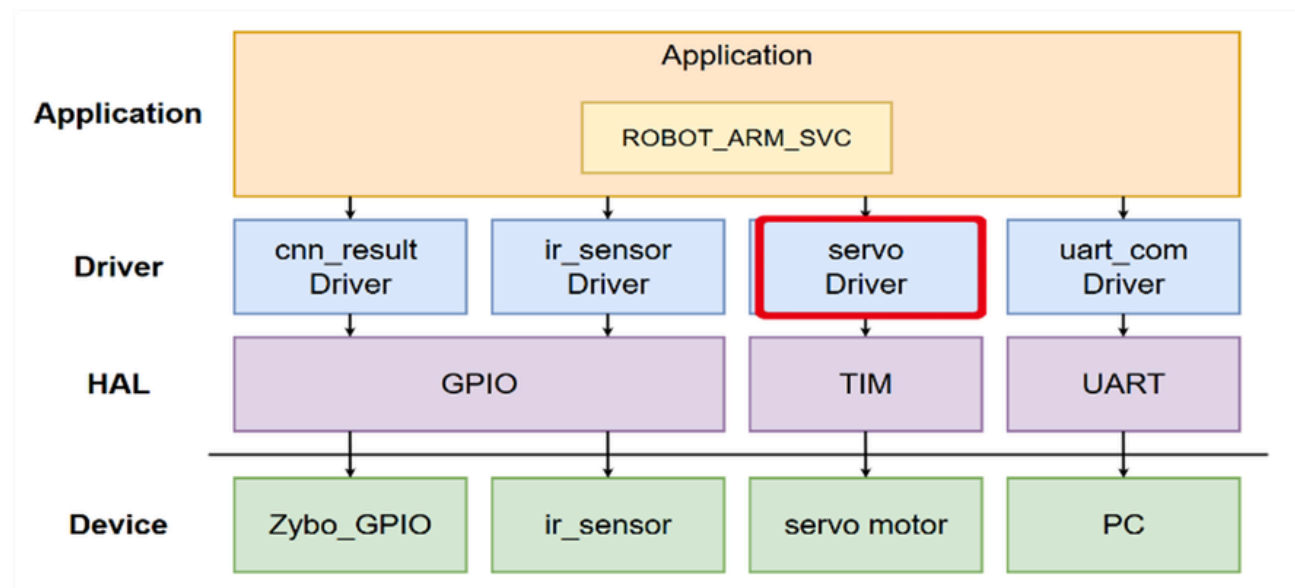
CNN Result

Zybo FPGA에서 GPIO 2핀(PC0, PC1)으로 전달되는 CNN 분류 결과를
수신, 저장, 제공하는 드라이버



<Robot arm service 로직 구성>

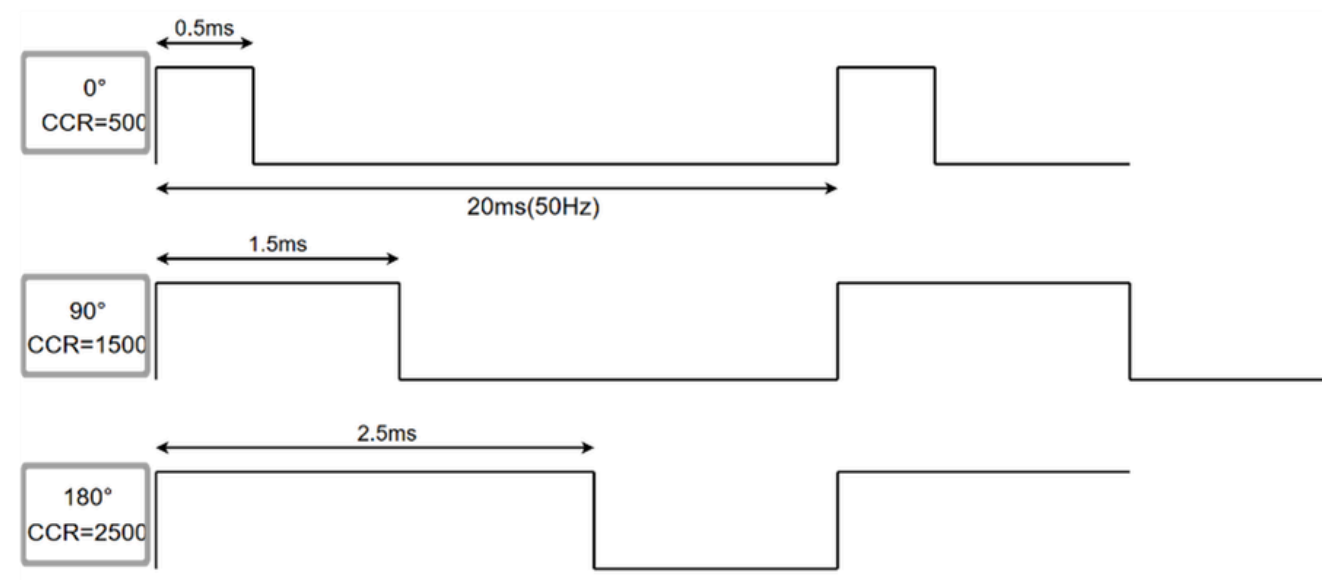
로봇팔 제어



Servo Motor Driver

PWM 신호의 펄스 폭으로 각도를 제어.
50Hz 기준으로 CCR 값이 펄스 폭을 결정.

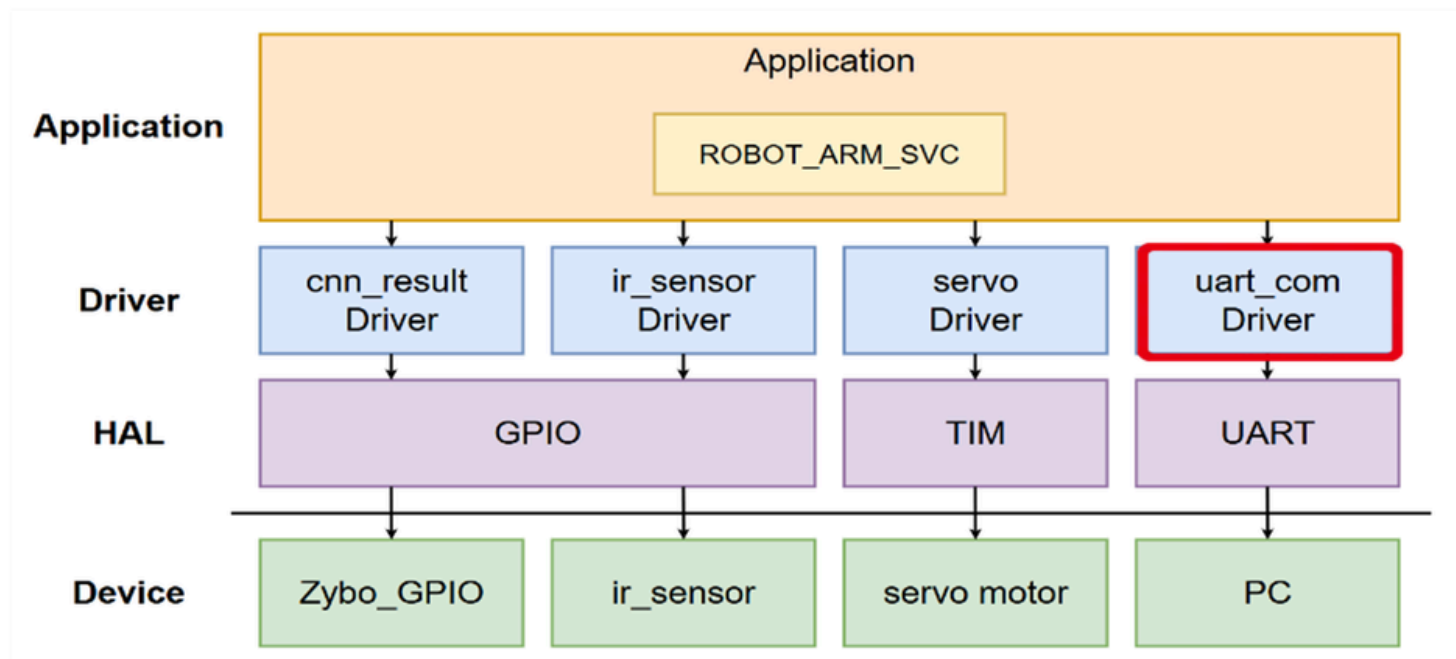
여기서 CCR(Capture/Compare Register)이란?
→ PWM 신호의 High 구간(Pulse Width) 길이를 결정하는 레지스터.
서보 모터의 제어 각도를 설정.



$$CCR = 500 + (\text{angle}/180) * 2000$$

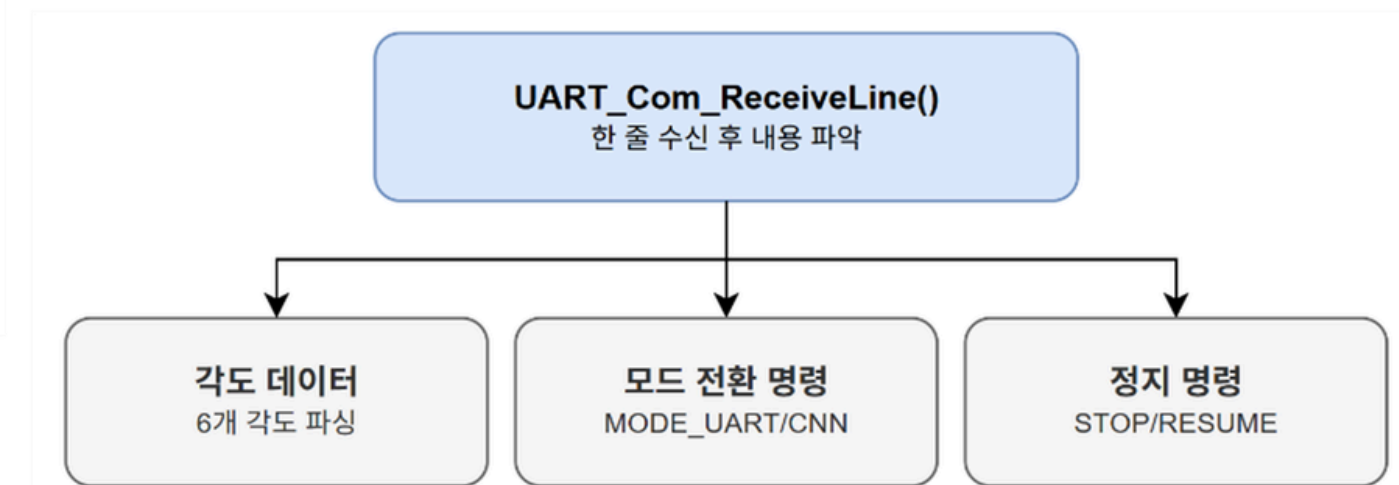
목표 각도	High 구간(ms)	CCR 설정값
0°	0.5ms	500
90°	1.5ms	1500
180°	2.5ms	2500

로봇팔 제어



UART Driver

STM32와 외부 장치(Python PC, 라즈베리파이) 간 UART 통신 프로토콜 담당.

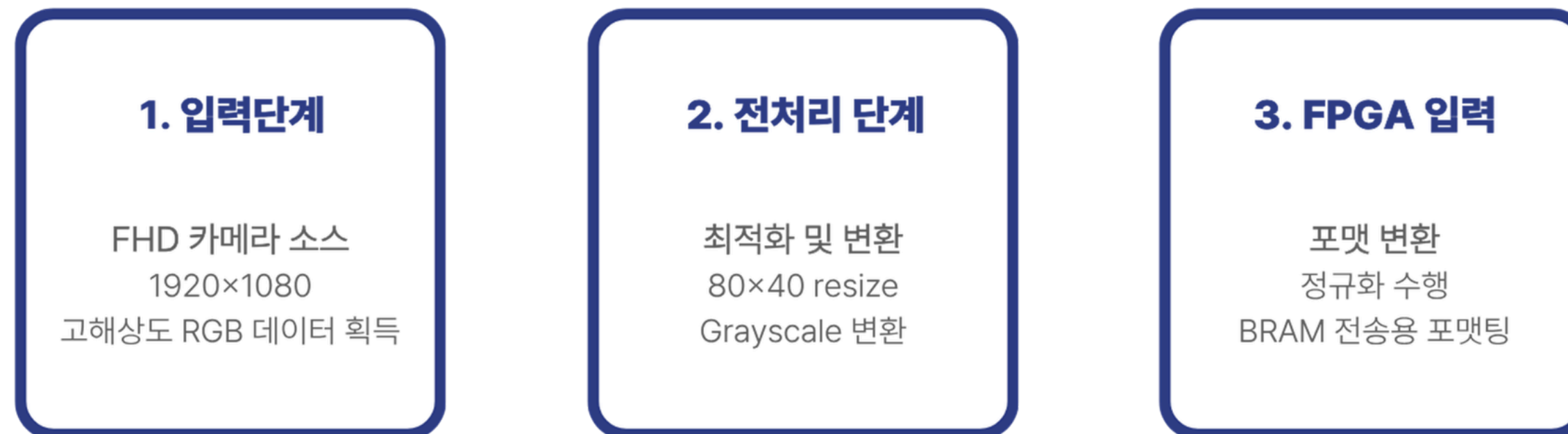


<UART_COM 로직 구성>

CNN 카메라 인식



파이프라인



카메라 CNN

왜 CNN으로 이미지 분석을 하는가?

지역 특징 추출

인접한 픽셀 간의 관계를 분석
특징을 효과적으로 포착

위치 불변성

특징이 어떤 위치에 있더라도
동일한 패턴으로 인식

파라미터 효율성

필터를 공유해 파라미터 수를
대폭 줄여 연산 효율 증가

FPGA 보드로 구현한 이유는?

저전력 소비

순수 하드웨어 로직만 사용
전력 효율 극대화

지연 시간

하드웨어에서 즉시 처리되어
일관된 실시간 성능 보장

유연한 재구성

새 알고리즘이 등장해도
로직 업데이트로 대응 가능

병렬 처리 최적화

DSP를 동시에 활용하여
합성곱 연산을 병렬 처리

Convolution

Kernel/Filter

지역적 특징을 계산하는 행렬

Padding & Stride

출력 크기를 조절, 필터의 이동 간격을 결정

Source

5	2	6	8	2	0	1	2
4	3	4	5	1	9	6	3
3	9	2	4	7	6	9	1
9	3	4	7	3	3	2	2
9	7	6	8	9	5	8	5
8	9	5	6	4	7	8	6
5	7	9	7	7	9	7	9
7	5	7	3	8	8	5	4

Kernel

-1	0	1
2	1	2
1	-2	0

Destination

$$\begin{aligned} &(-1 \times 5) + (0 \times 2) + (1 \times 6) + \\ &(2 \times 4) + (1 \times 3) + (2 \times 4) + \\ &(1 \times 3) + (-2 \times 9) + (0 \times 2) = 5 \end{aligned}$$

BatchNorm

계층적 정규화

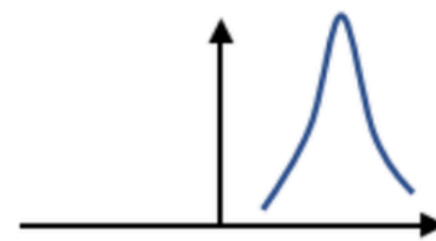
미니배치 단위로
평균 0, 분산 1이 되도록
정규화

학습 가능 파라미터

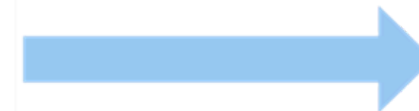
정규화 후 감마와 베타를 사용
데이터 표현력을 유지

강력한 최적화

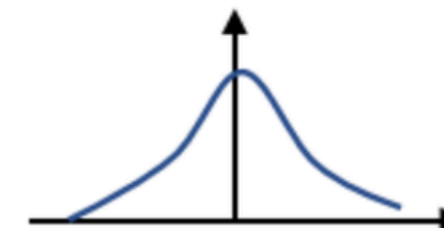
기울기 흐름을 개선하여
더 높은 학습률 사용 가능



기존 데이터 분포



BatchNorm 계산



평균 0, 분산 1로 분포

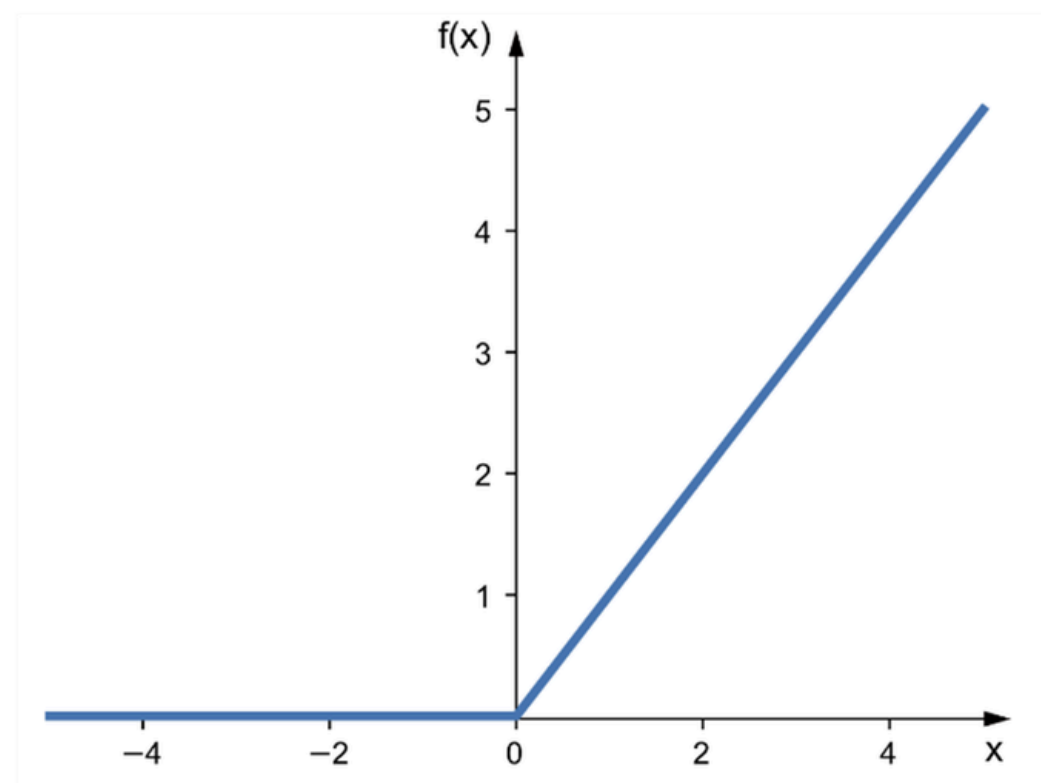
단순히 데이터를 정리하는 것을 넘어, 신경망이 깊어져도 안정적으로 학습할 수 있음

ReLU

Convolution 연산 결과에 비선형성을 부여하여, 복잡한 패턴을 학습 가능

비선형성이란? 입력과 출력이 직선 관계가 아닌 것

-> 직선함수만 사용하면 층을 아무리 쌓아도 결국 하나의 큰 직선이 되기에, 곡선을 만들어 이미지를 구현하기 위해서는 ReLU가 필요



0보다 작으면 무조건 0 (차단), 0보다 크면 그대로 (통과)
이 꺾이는 성질이 여러 층 겹칠 때, 복잡한 곡선 표현 가능

Example

1	14	-9	4
-2	-20	10	6
-3	3	11	1
2	54	-2	80

ReLU

1	14	0	4
0	0	10	6
0	3	11	1
2	54	0	80

Rectified Linear Unit

Pooling

데이터의 공간적인 크기를 줄여 연산량을 감소

다운샘플링

연산량을 감소시키고 주요 정보를 요약

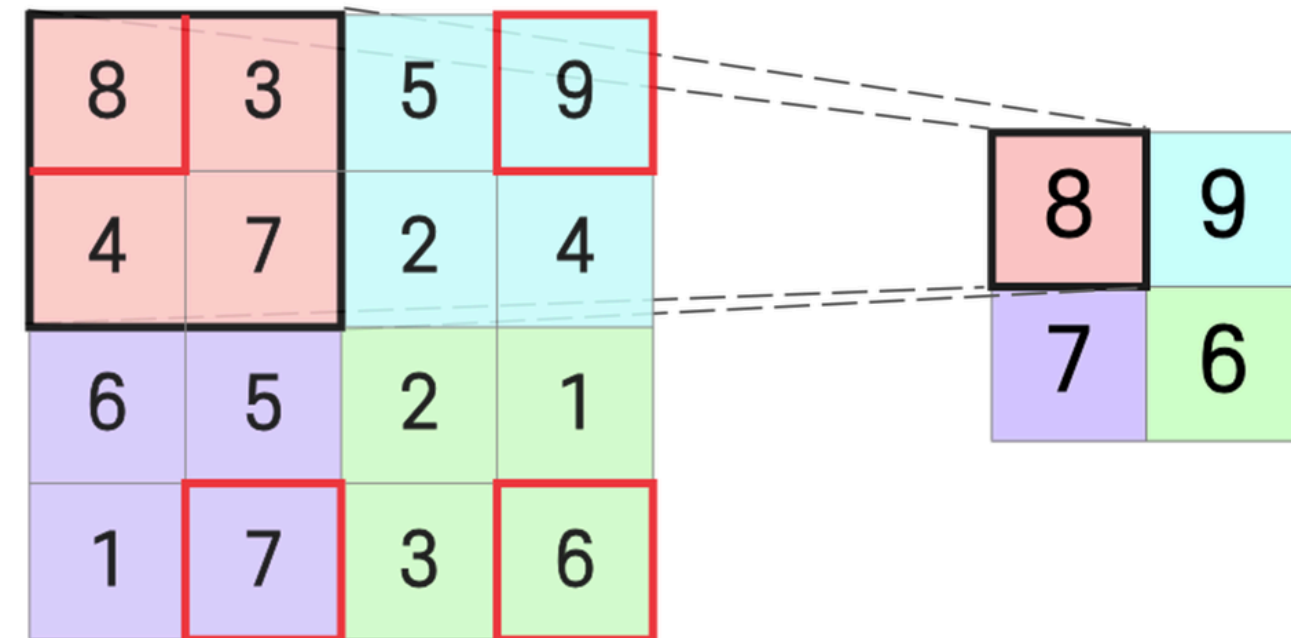
Max Pooling

특정 영역 내에서 가장 큰 값을 선택

과적합 방지

모델의 일반화 성능 보장

Example



Fully Connected

CNN 모델의 뒷부분에서 최종 판단을 내리는 두뇌 역할

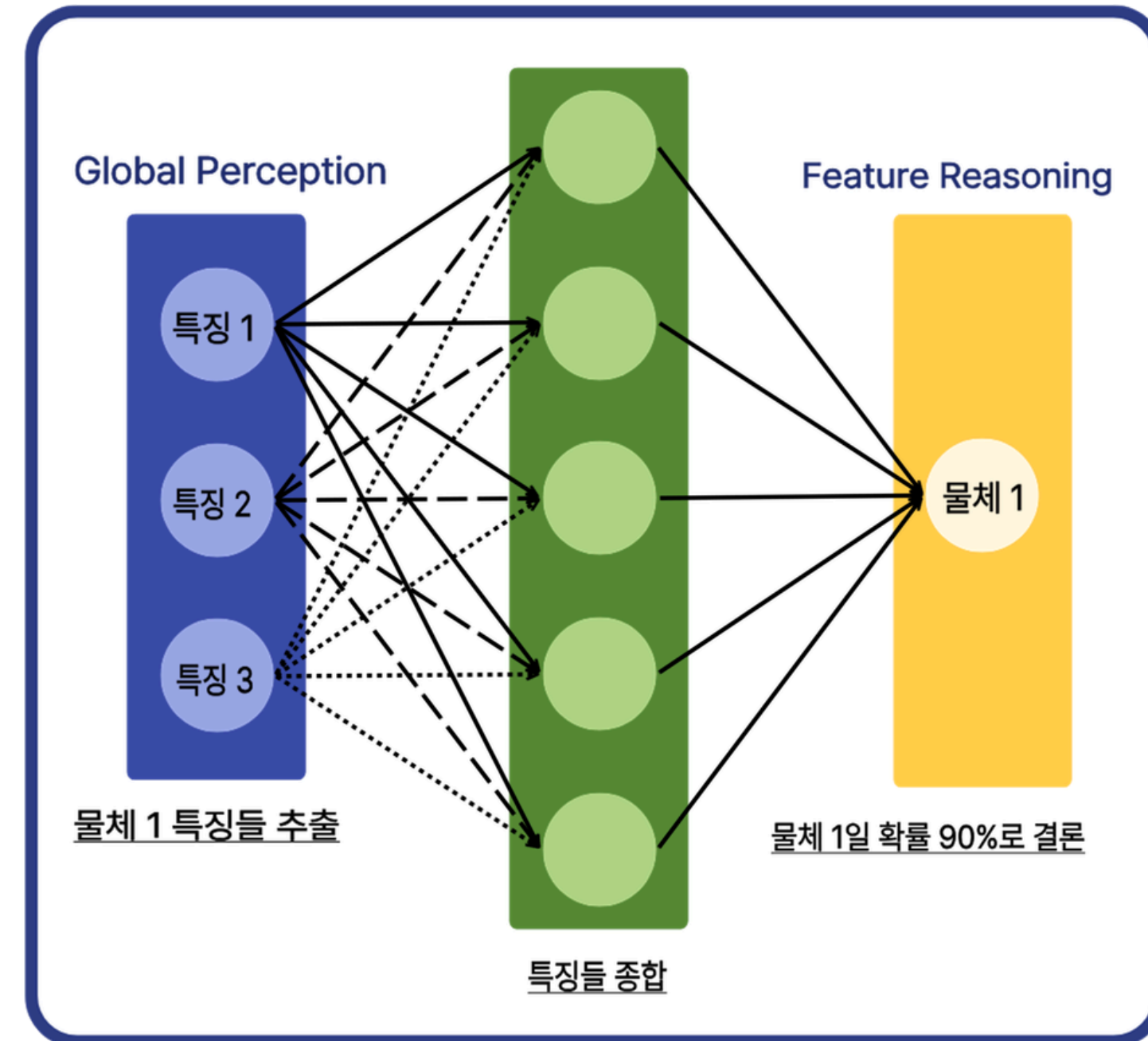
이전 레이어의 모든 출력이
현재 레이어의 모든 뉴런과 연결되는 구조

Global Perception

지역적 패턴이 아닌 전체적인 특징을 통합

Feature Reasoning

추출된 특징들을 조합하여 최종 결과를 추론



CNN 모델 학습



XK00

정상 차량 1



MW00

정상 차량 2



RZ00

정상 차량 3



OP00

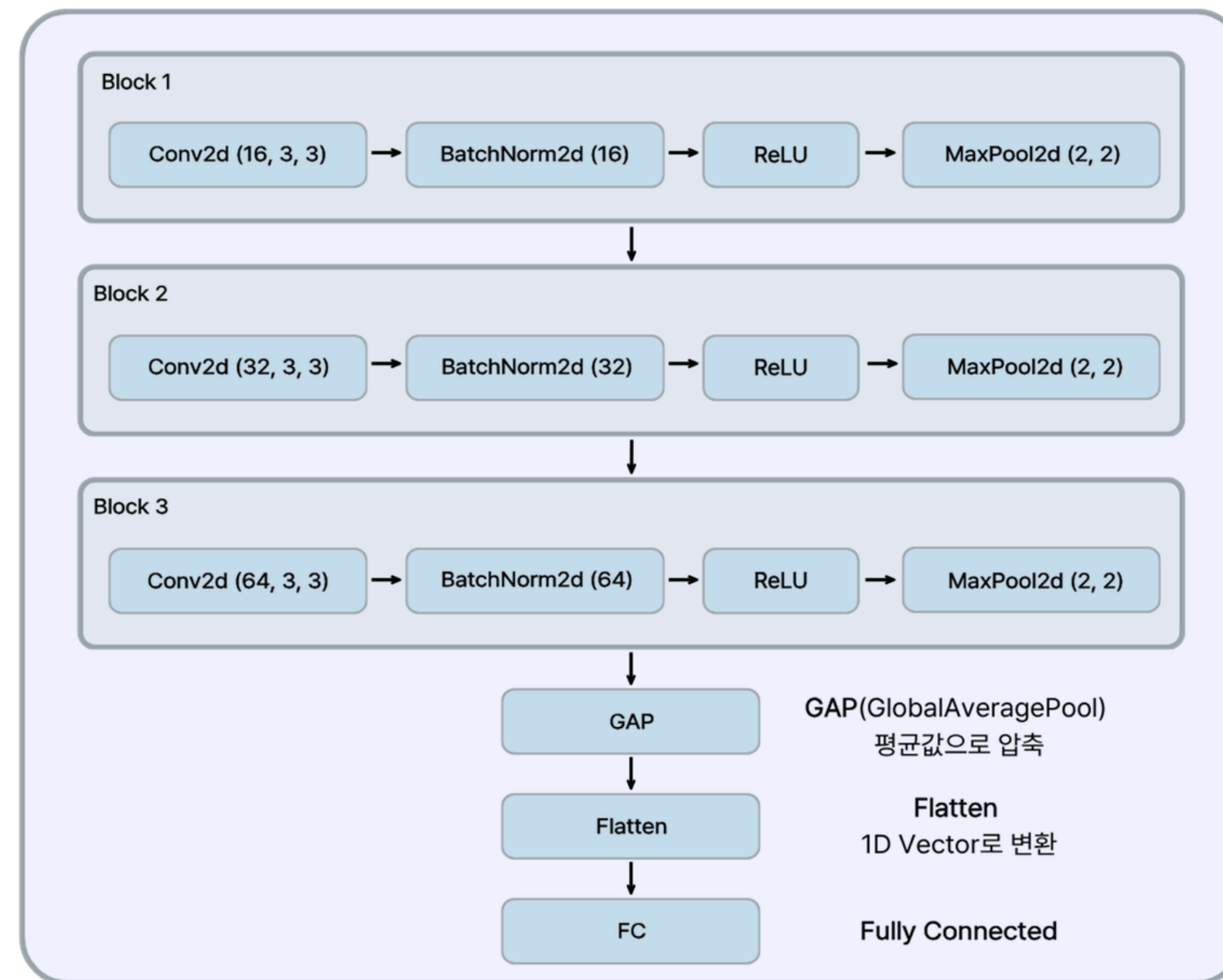
불량 차량

'XK', 'MW', 'RZ', 'OP' 그룹으로 분류

CNN 모델 학습

연산 최적화
3x3 합성곱을 사용하여 파라미터 최소화

공간 정보 압축
단계적 풀링을 통한 특징 요약



CNN 모델 학습

주요 레이어	커널 / 필터	입력 크기	출력 크기	파라미터 수
Layer 1	16 Filters (3×3)	1 × 40 × 80	16 × 20 × 40	176
Layer 2	32 Filters (3×3)	16 × 20 × 40	32 × 10 × 20	4,672
Layer 3	64 Filters (3×3)	32 × 10 × 20	64 × 5 × 10	18,560
분류 헤드 (FC)	4 Classes	64 (GAP)	4 (Logits)	260

저용량 메모리 -> 약 0.1 MB 미만의 모델 크기
고속 추론 -> 적은 연산량으로 실시간 처리 가능
FPGA 적합성 -> 리소스가 한정된 하드웨어에 최적화

↓
23,668
총 파라미터 수

CNN 모델 학습



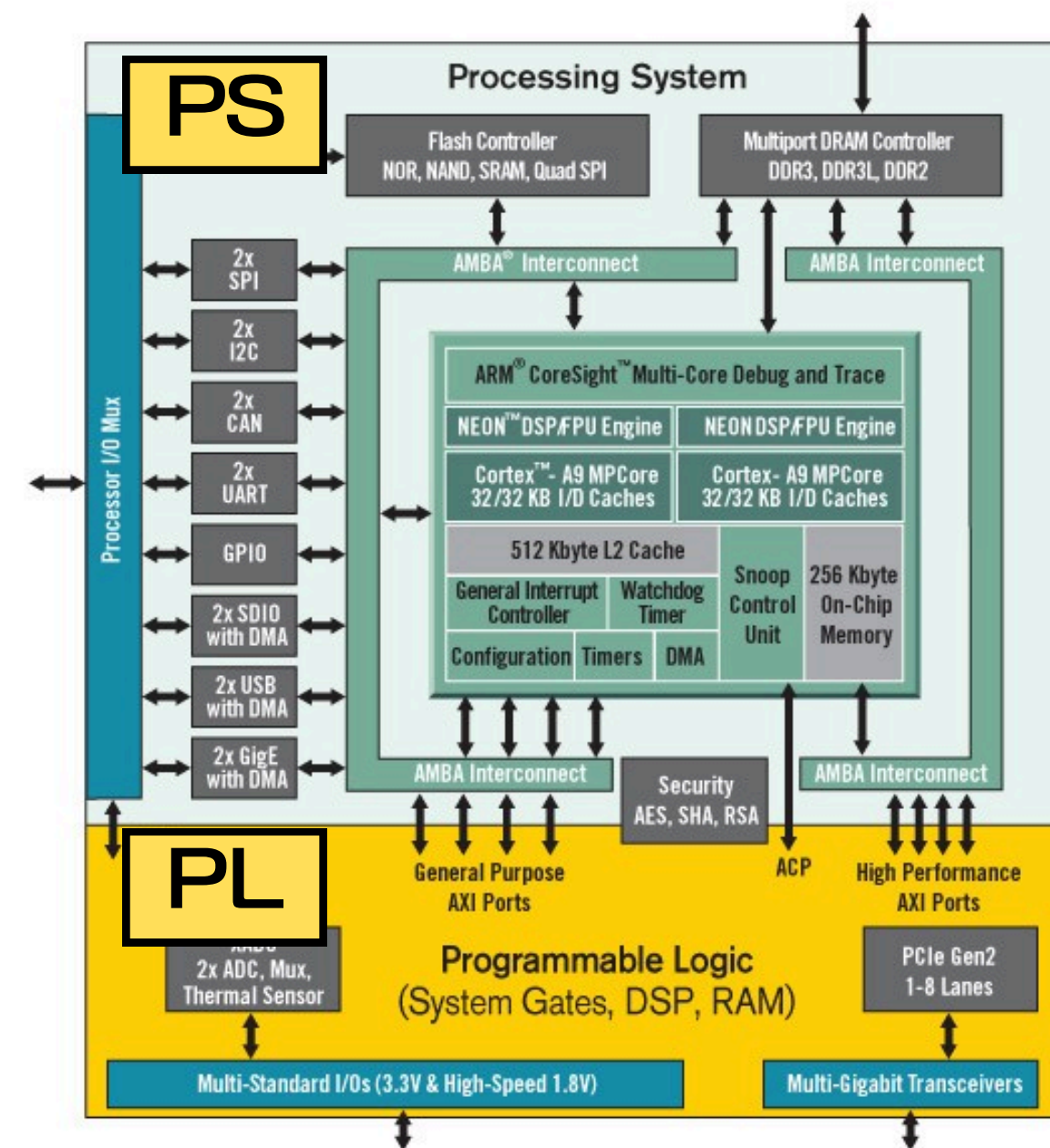
기본 폰트 기반 합성 데이터 생성
모델 일반화를 위한 광학적/기하학적 증강 적용
총 28,000개의 고해상도 학습 이미지 확보

CNN 카메라 인식

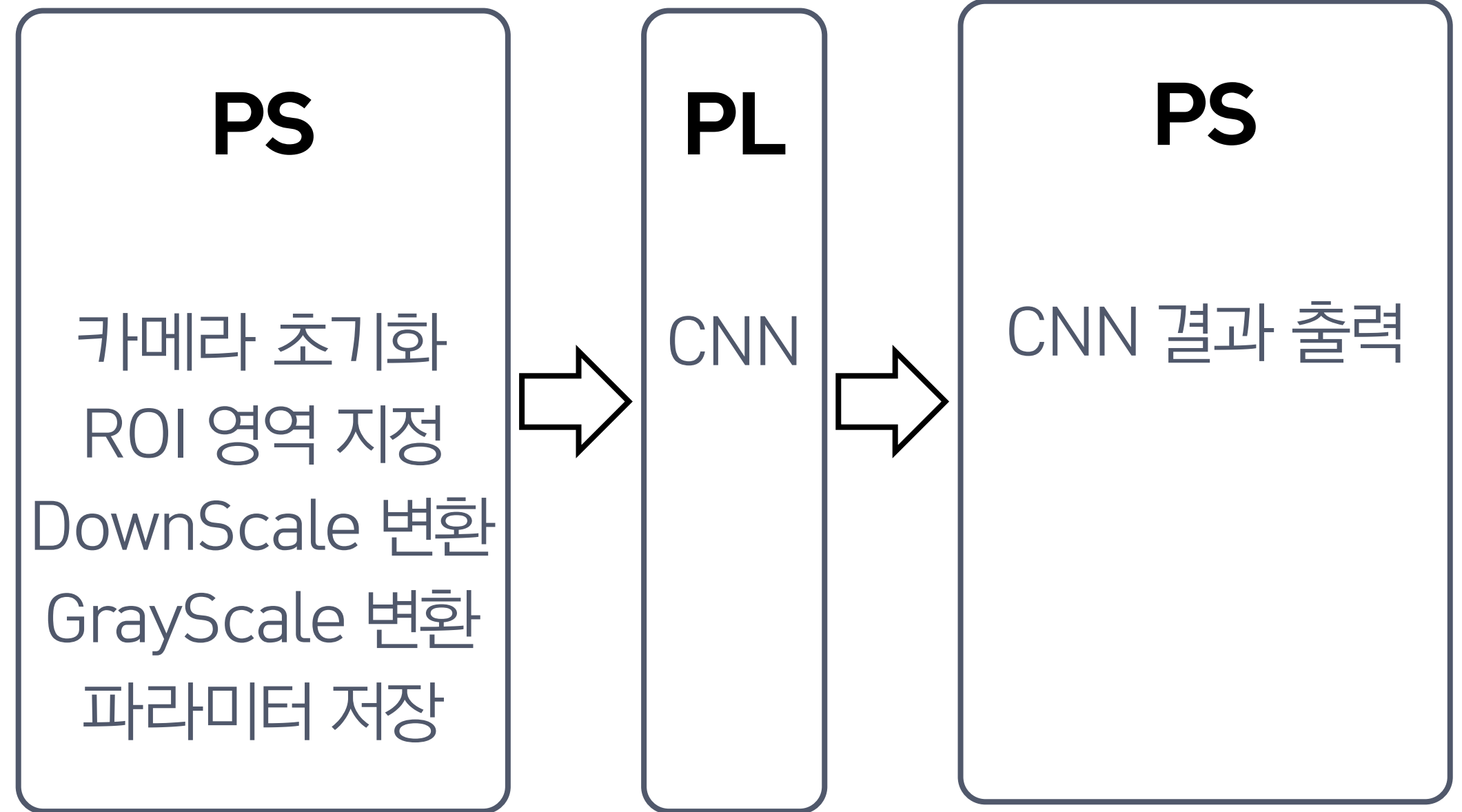
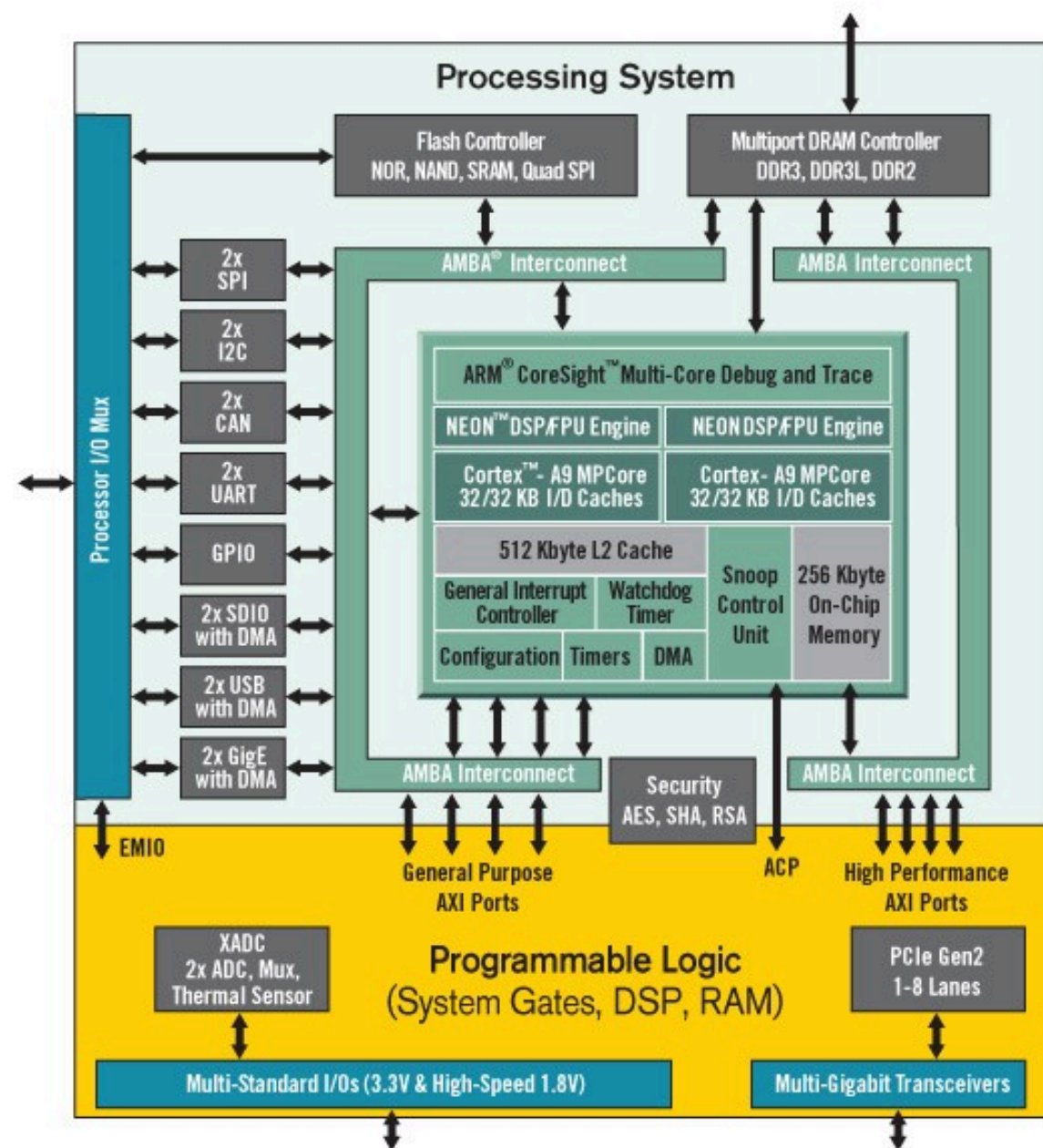


ROI 적용

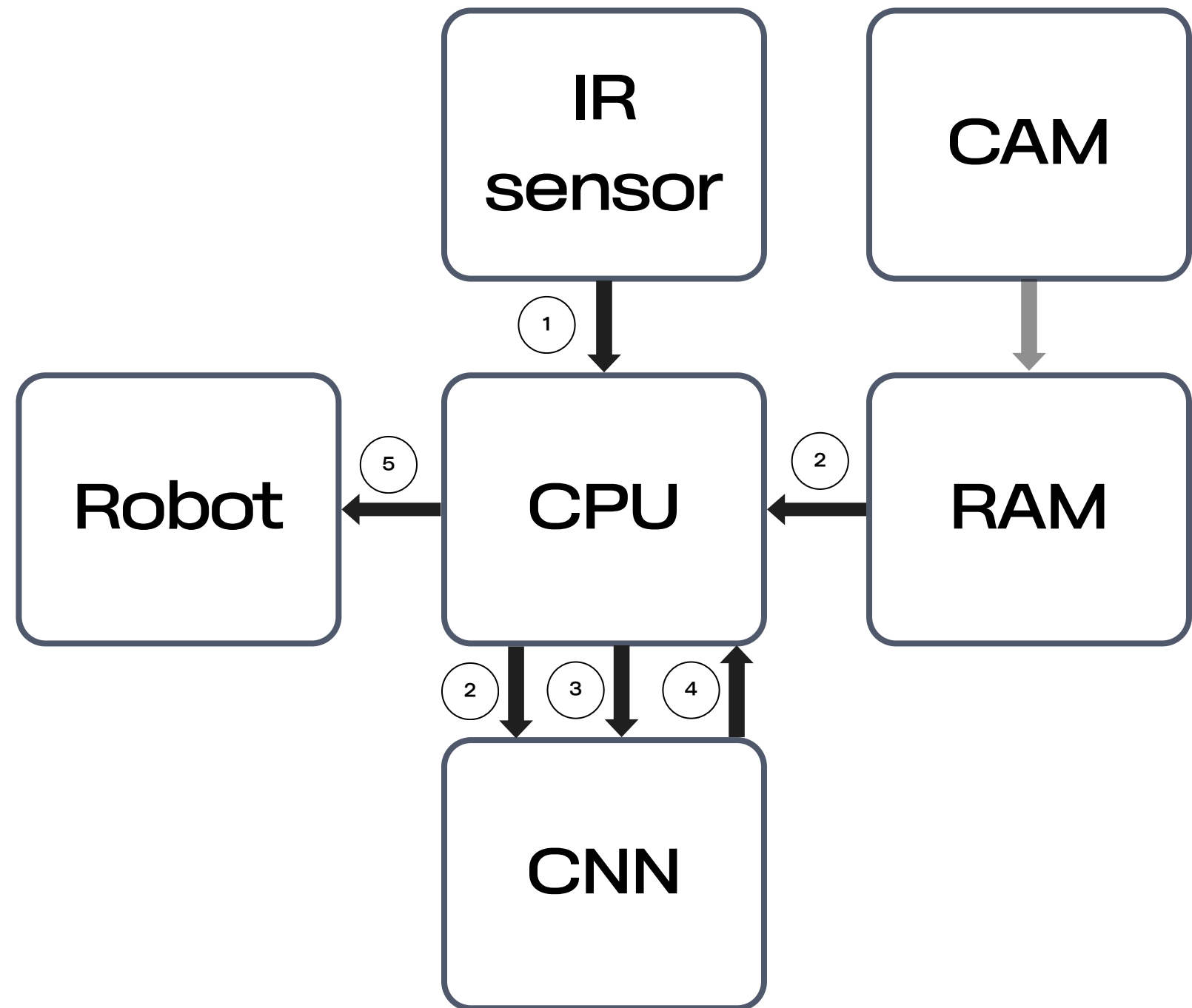
Zybo Z7-20 기반 제어 구조



Zybo Z7-20 기반 제어 구조



VITIS 기반 CNN 제어 흐름



1. 적외선 신호 입력
2. 입력 데이터 전달
3. CNN start
4. 추론 결과 확인
5. 추론 결과 로봇팔에 전달

저전력 FPGA 추론 결과

Processing Time
처리 시간

약 **1.8배 ↑**

POWER
전력 소비

약 **27배 ↓**

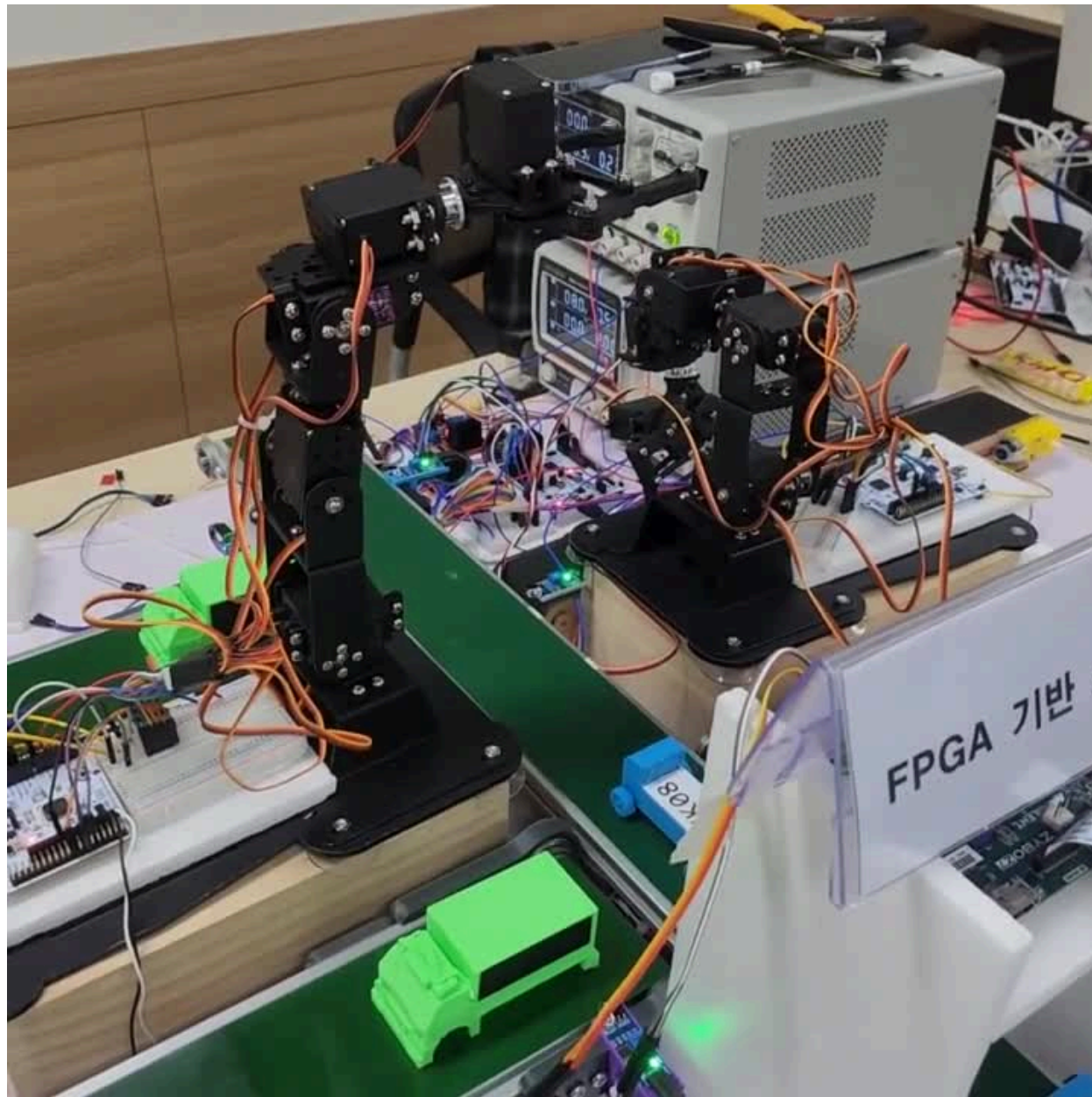
ACCURACY
분류 정확도

약 **~95.6%**

※ GPU 대비 성능

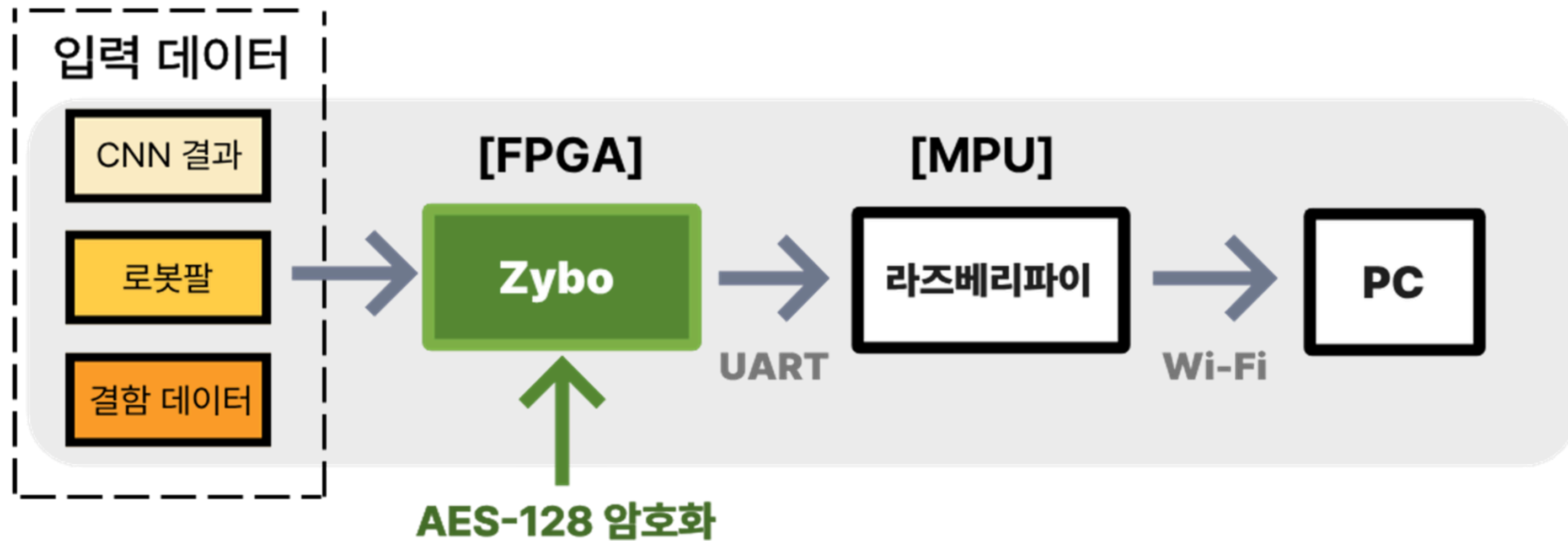
비교 항목	GPU (RTX4070)	본 프로젝트 FPGA (Zybo z7-20)	비고
지연 시간(ms)	약 30ms	약 55ms	평균 추론 속도 (10 Frame 당)
소비 전력(W)	51.81 W	1.93 W	추론 중 평균 소비 전력
평균 정확도(%)	100%	95.6%	20,000장 기준 GPU 대비 FPGA 정확도 (*양자화 기준)

동작 영상

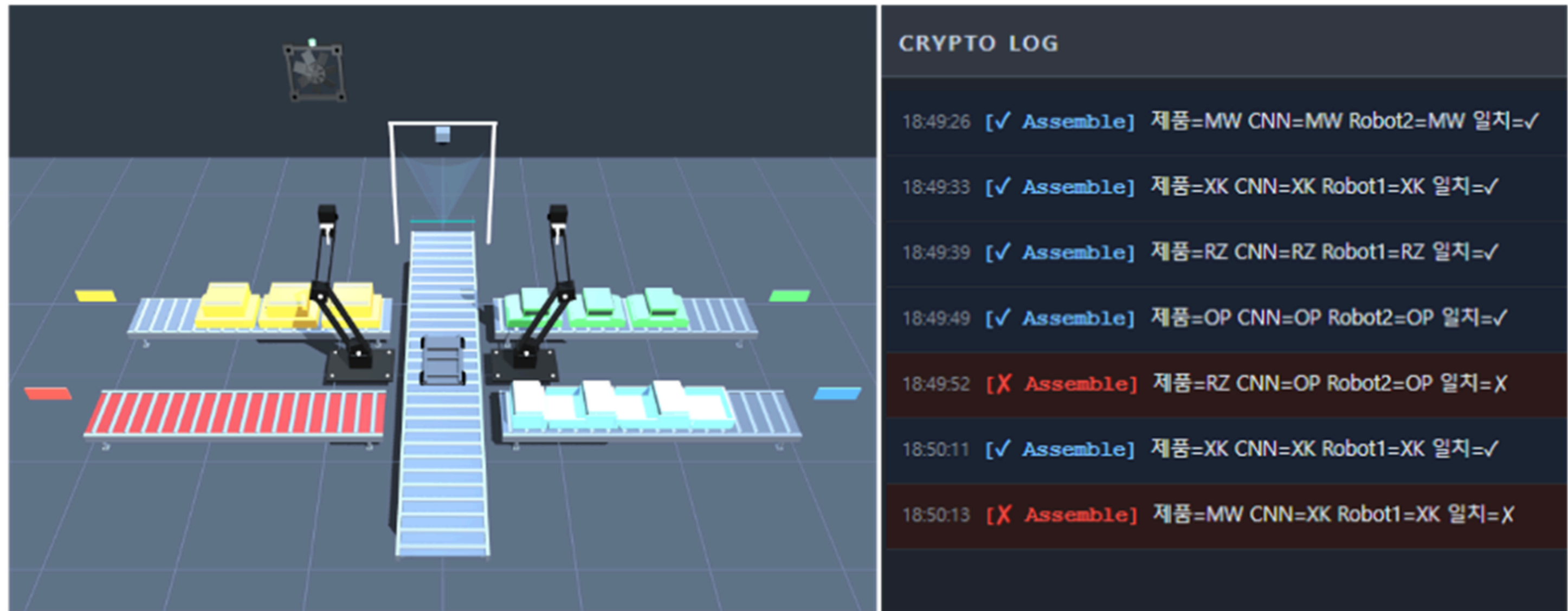


1. 카메라: 일련번호 추론
2. 컨베이어 벨트: 차량 이동
3. 적외선 센서: 차량 인식
4. 로봇팔: 조립 동작

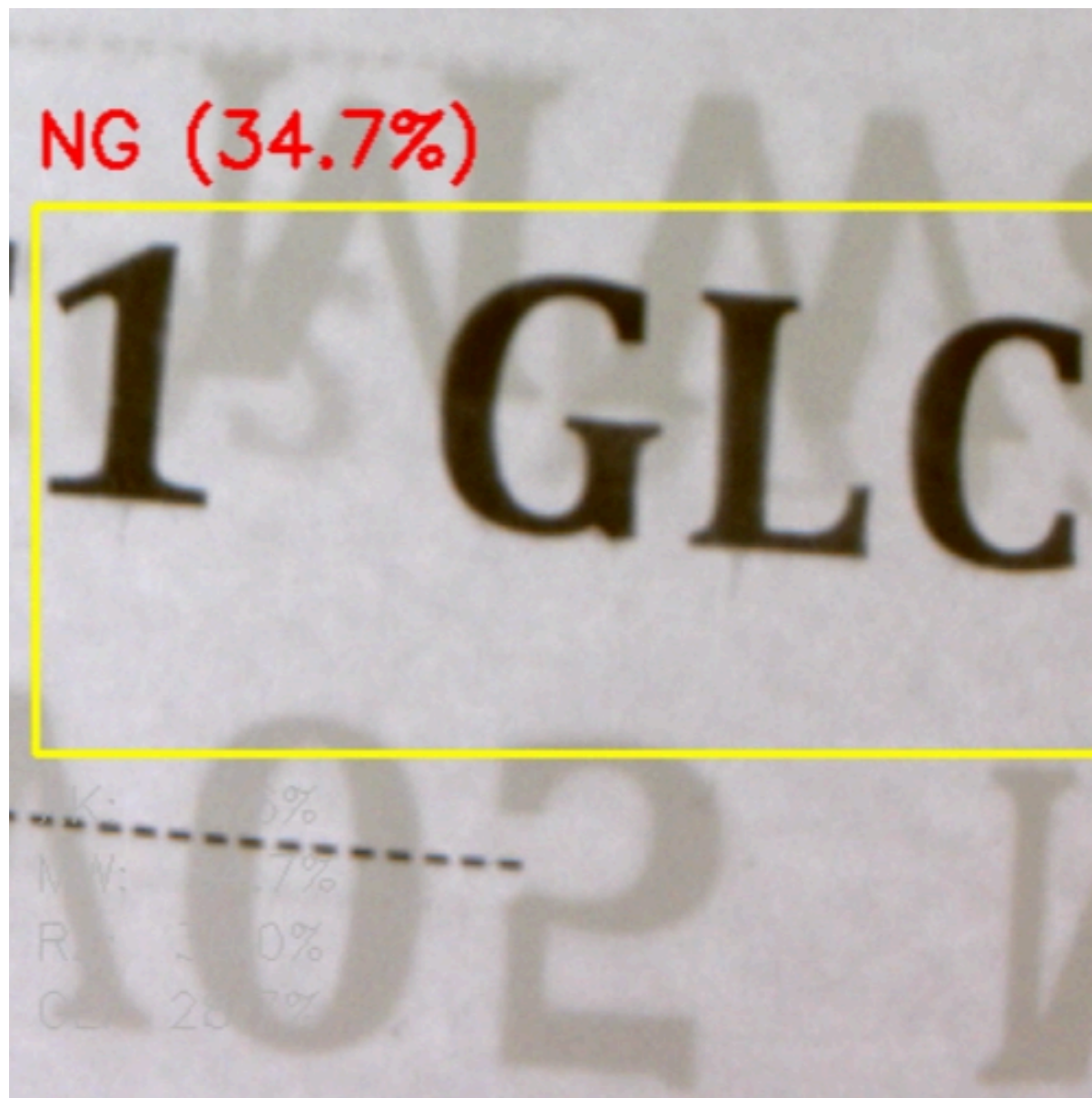
서버 전송



GUI 통합



Troubleshooting



CNN을 시작할 때, ROI영역이 글자를 담지 못하면 인식률이 떨어지는 문제 발생

→ 적외선 센서에 차량 하체가 도달했을 때 CNN을 시작하도록 센서의 위치와 ROI영역 수정

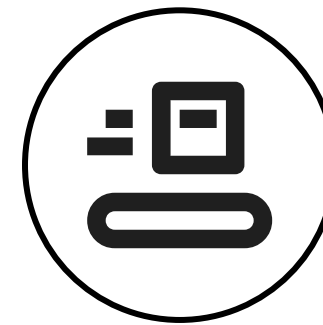
→ ROI영역에서 벗어나도 인식이 가능하도록 추가 데이터 학습

펌웨어 개발을 통해 얻은 역량



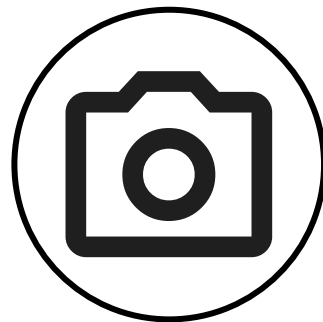
Sensor Input

센서 입력을 기준으로 장치 동작을 제어하는 흐름 경험



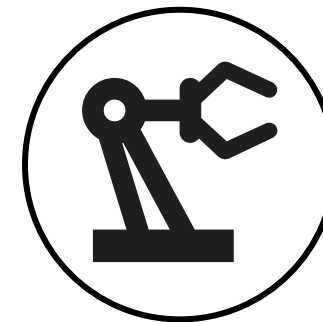
Process Control

컨베이어, 카메라, 로봇팔을 하나의 공정으로 연동한 경험



CNN Decision

CNN 결과를 실제 제어 명령으로 변환하는 구조 이해



Device Output

스마트팩토리에서 입력, 판단, 출력, 기록이 연결되는 구조 이해